



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Course: Computer Vision

Course Code: ITA0506

Slot: A

Faculty: Jeni Gracia R J

Submitted By

Y Sravan Kumar (192472289)



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

LAB PROGRAMS

Experiment No. 1: Basic Image Handling and Image Processing using OpenCV

Aim

To read an image using OpenCV in Python and convert the image into a grayscale image.

Software Requirements

Python 3.x

OpenCV Library (opencv-python)

Procedure

1. Import the OpenCV library.
2. Read the input image using cv2.imread().
3. Check whether the image is loaded successfully.
4. Convert the colour image into a grayscale image using cv2.cvtColor().
5. Display both the original and grayscale images.
6. Wait for a key press and close all image windows.

Code

```
Exp1.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/CV Exp 1.py/Exp1.py (3.12.3)
File Edit Format Run Options Window Help
import cv2

# Read the image
image = cv2.imread("sample.jpg")

# Check whether image is loaded
if image is None:
    print("Error: Image not found!")
else:
    # Display original image
    cv2.imshow("Original Image", image)

    # Convert image to grayscale
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # Display grayscale image
    cv2.imshow("Grayscale Image", gray)

    # Wait until any key is pressed
    cv2.waitKey(0)

    # Close all windows
    cv2.destroyAllWindows()
```

Output



Result

The image was successfully read using OpenCV and converted from a color image to a grayscale image. Both the original and grayscale images were displayed successfully.

Experiment No. 2: Image Blurring using Gaussian Blur in OpenCV

Aim

To read an image in Python and apply Gaussian Blur using OpenCV.

Software Requirements:

- Python 3.x
- OpenCV (cv2)
- NumPy
- Visual Studio Code (VS Code)

Procedure

1. Import the OpenCV library.
2. Read the input image using `cv2.imread()`.
3. Apply Gaussian Blur using `cv2.GaussianBlur()`.
4. Display the original and blurred images.
5. Save the blurred image (optional).

Code:

```
Guassinblur 2.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/Cv Exp 2.py/Guassinblur 2.py (3.12.3)
File Edit Format Run Options Window Help
import cv2

# Read the image
image = cv2.imread("sample1.jpg")

# Check if image is loaded successfully
if image is None:
    print("Error: Image not found!")
else:
    # Apply Gaussian Blur
    blurred = cv2.GaussianBlur(image, (15, 15), 0)

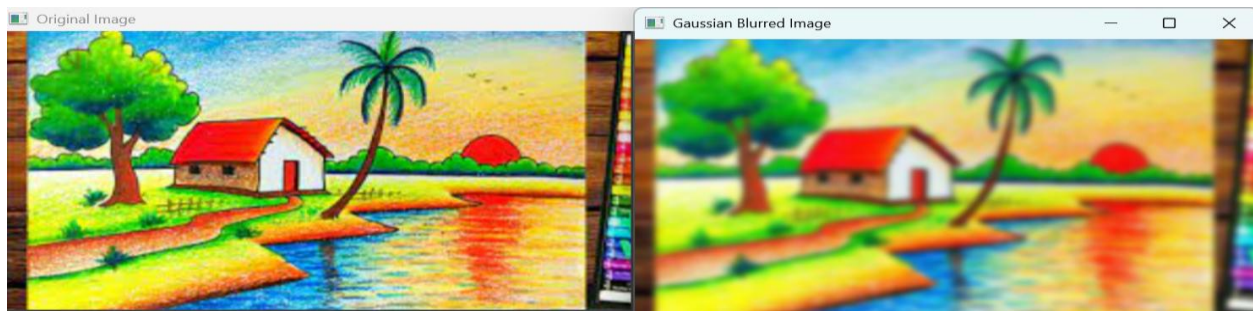
    # Display Original Image
    cv2.imshow("Original Image", image)

    # Display Blurred Image
    cv2.imshow("Gaussian Blurred Image", blurred)

    # Wait for a key press
    cv2.waitKey(0)

    # Close all windows
    cv2.destroyAllWindows()
```

Output:



Result

The image was successfully blurred using the Gaussian Blur technique in OpenCV.

Experiment No. 3: Edge Detection using Canny Function in OpenCV

Aim

To read an image in Python and detect its edges using the **Canny** edge detection function in OpenCV.

Software Requirements:

- Python 3.x
- OpenCV (cv2)
- NumPy
- Visual Studio Code (VS Code)

Procedure

1. Import the OpenCV library.
2. Read the input image using cv2.imread().
3. Convert the image to grayscale.
4. Apply the cv2.Canny() function to detect edges.
5. Display the original and edge-detected images.

Code:

```
Canny Edge Detection Function.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/cv exp3.py/Canny Edge Detection Function.py (3.12.3)
File Edit Format Run Options Window Help
import cv2

# Read the image
image = cv2.imread("sample2.jpg")

# Check if image is loaded
if image is None:
    print("Error: Image not found!")
else:
    # Convert image to grayscale
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # Apply Canny Edge Detection
    edges = cv2.Canny(gray, 100, 200)

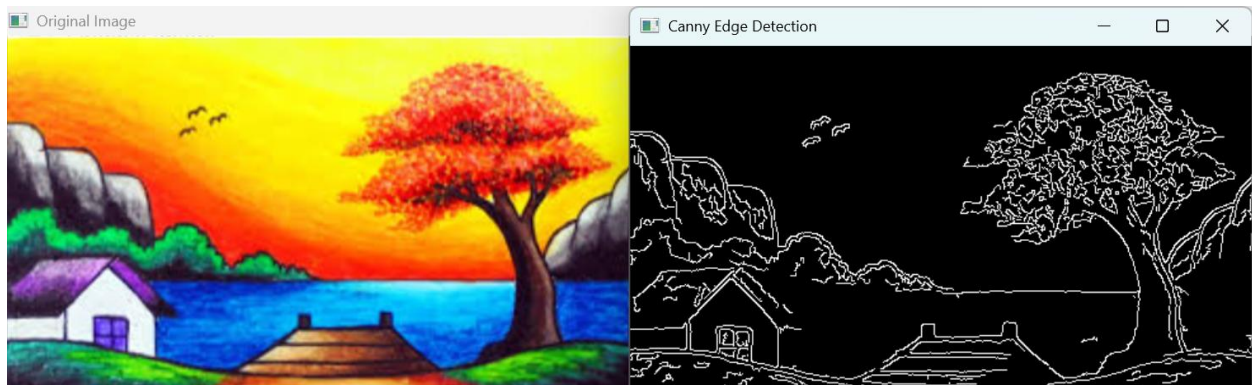
    # Display original image
    cv2.imshow("Original Image", image)

    # Display edge-detected image
    cv2.imshow("Canny Edge Detection", edges)

    # Wait for a key press
    cv2.waitKey(0)

    # Close all windows
    cv2.destroyAllWindows()
```

Output:



Result

The image was successfully processed, and its edges were detected using the Canny edge detection technique in OpenCV.

Experiment No. 4: Image Dilation using Dilate Function in OpenCV

Aim

To read an image in Python and apply the **Dilate** function using OpenCV.

Software Requirements:

- Python 3.x
- OpenCV (cv2)
- NumPy
- Visual Studio Code (VS Code)

Procedure

1. Import the OpenCV library.
2. Read the input image using `cv2.imread()`.
3. Create a kernel for dilation.
4. Apply the `cv2.dilate()` function to the image.
5. Display the original and dilated images.

Code:

```
Dilate Function.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/cv exp4.py/Dilate Function.py (3.12.3)
File Edit Format Run Options Window Help
import cv2
import numpy as np

# Read the image
image = cv2.imread("sample3.jpg")

# Check if image is loaded
if image is None:
    print("Error: Image not found!")
else:
    # Create a kernel
    kernel = np.ones((5, 5), np.uint8)

    # Dilate the image
    dilated = cv2.dilate(image, kernel, iterations=1)

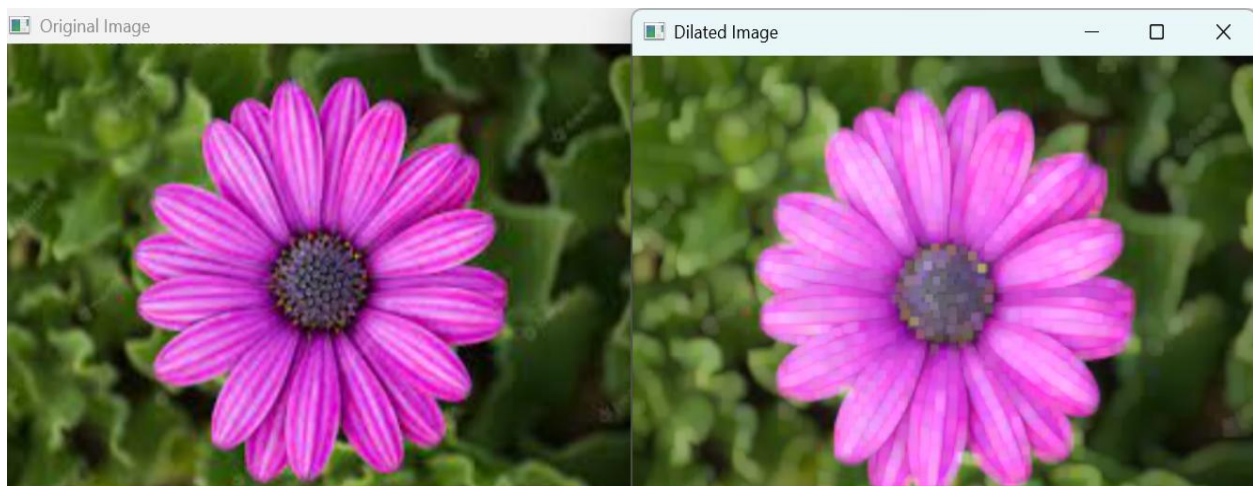
    # Display original image
    cv2.imshow("Original Image", image)

    # Display dilated image
    cv2.imshow("Dilated Image", dilated)

    # Wait for a key press
    cv2.waitKey(0)

    # Close all windows
    cv2.destroyAllWindows()
```

Output:



Result

The image was successfully processed, and the objects in the image were enlarged using the Dilate function in OpenCV.

Experiment No. 5: Image Erosion using Erode Function in OpenCV

Aim

To read an image in Python and apply the Erode function using OpenCV.

Software Requirements:

- Python 3.x
- OpenCV (cv2)
- NumPy
- Visual Studio Code (VS Code)

Procedure

1. Import the OpenCV library.
2. Read the input image using cv2.imread().
3. Create a kernel for erosion.
4. Apply the cv2.erode() function to the image.
5. Display the original and eroded images.

Code:

```
Erode Function.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/cv exp 5.py/Erode Function.py (3.12.3)
File Edit Format Run Options Window Help
import cv2
import numpy as np

# Read the image
image = cv2.imread("sample5.jpg")

# Check if image is loaded
if image is None:
    print("Error: Image not found!")
else:
    # Create a kernel
    kernel = np.ones((5, 5), np.uint8)

    # Erode the image
    eroded = cv2.erode(image, kernel, iterations=1)

    # Display original image
    cv2.imshow("Original Image", image)

    # Display eroded image
    cv2.imshow("Eroded Image", eroded)

    # Wait for a key press
    cv2.waitKey(0)

    # Close all windows
    cv2.destroyAllWindows()
```

Output:



Result

The image was successfully processed, and the objects in the image were reduced using the Erode function in OpenCV.

Experiment No. 6: Basic Video Processing (Slow Motion and Fast Motion)

Aim

To read a captured video in Python using OpenCV and display it in slow motion and fast motion.

Software Requirements

- Python 3.x
- OpenCV (cv2)
- Visual Studio Code (VS Code)
- Captured video file (e.g., sample.mp4)

Procedure

1. Import the OpenCV library.
2. Load the captured video using `cv2.VideoCapture()`.

3. Read the video frame by frame.
4. Display the video in slow motion by increasing the delay.
5. Display the video in fast motion by decreasing the delay.
6. Press 'q' to exit the video.

Code

```
videoprocessing.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/Scripts/Python.exe
File Edit Format Run Options Window Help
import cv2

video = cv2.VideoCapture("sample.mp4")

while video.isOpened():
    ret, frame = video.read()

    if not ret:
        break

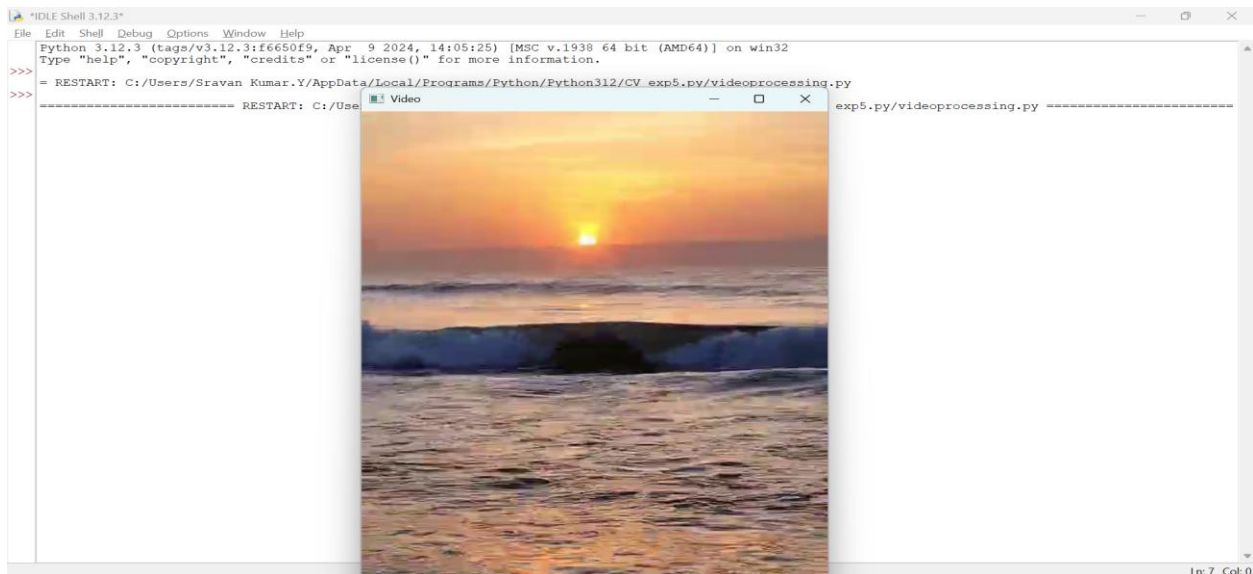
    cv2.imshow("Video", frame)

    # Change delay:
    # 100 = Slow Motion
    # 10 = Fast Motion

    if cv2.waitKey(10) & 0xFF == ord('q'):
        break

video.release()
cv2.destroyAllWindows()
```

Output:



Experiment No. 7: Capture Video from Webcam and Display in Slow Motion and Fast Motion

Aim

To capture video from a webcam in Python and display it in slow motion and fast motion using OpenCV.

Software Requirements

- Python 3.x
- OpenCV (cv2)
- Visual Studio Code (VS Code)
- Webcam

Procedure

1. Import the OpenCV library.
2. Access the webcam using `cv2.VideoCapture(0)`.
3. Capture video frame by frame.
4. Display the captured video.
5. Use `cv2.waitKey(100)` for slow motion or `cv2.waitKey(10)` for fast motion.
6. Press 'q' to stop the video capture.

Code

```
import cv2
```

```
cap = cv2.VideoCapture(0)
```

```
while True:
```

```
    ret, frame = cap.read()
```

```
    if not ret:
```

```
        break
```

```
cv2.imshow("Webcam Video", frame)
```

```
# Slow Motion: 100
```

```
# Fast Motion: 10
```

```
if cv2.waitKey(100) & 0xFF == ord('q'):
```

```
    break
```

```
cap.release()
```

```
cv2.destroyAllWindows()
```

Output:

Result

The live video was successfully captured from the webcam and displayed in **slow motion** and **fast motion** using OpenCV in Python.

Experiment No. 8: Scaling an Image to Bigger and Smaller Sizes

Aim

To read an image in Python and scale it to bigger and smaller sizes using OpenCV.

Software Requirements

- Python 3.x
- OpenCV (cv2)
- NumPy
- Visual Studio Code (VS Code)

Procedure

1. Import the OpenCV library.
2. Read the input image using `cv2.imread()`.
3. Resize the image to a larger size using `cv2.resize()`.
4. Resize the image to a smaller size using `cv2.resize()`.
5. Display the original, enlarged, and reduced images.

Code

sacling.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/CV exp8.py/sacling.py (3.12.3)

File Edit Format Run Options Window Help

```
import cv2

img = cv2.imread("image.jpg")

# Bigger Image
bigger = cv2.resize(img, None, fx=2, fy=2)

# Smaller Image
smaller = cv2.resize(img, None, fx=0.5, fy=0.5)

cv2.imshow("Original Image", img)
cv2.imshow("Bigger Image", bigger)
cv2.imshow("Smaller Image", smaller)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output:



Result

The input image was successfully scaled to **bigger** and **smaller** sizes using the `cv2.resize()` function in OpenCV.

Experiment No. 9: Rotation of an Image in Clockwise and Counterclockwise Direction

Aim

To read an image in Python and rotate it in clockwise and counter clockwise directions using OpenCV.

Software Requirements

- Python 3.x
- OpenCV (cv2)
- NumPy
- Visual Studio Code (VS Code)

Procedure

1. Import the OpenCV library.
2. Read the input image using `cv2.imread()`.
3. Rotate the image clockwise using `cv2.rotate()`.
4. Rotate the image counter clockwise using `cv2.rotate()`.
5. Display the original and rotated images.

Code

```
rotation.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/CV exp9.py/rotation.py (3.12.3)
File Edit Format Run Options Window Help
import cv2

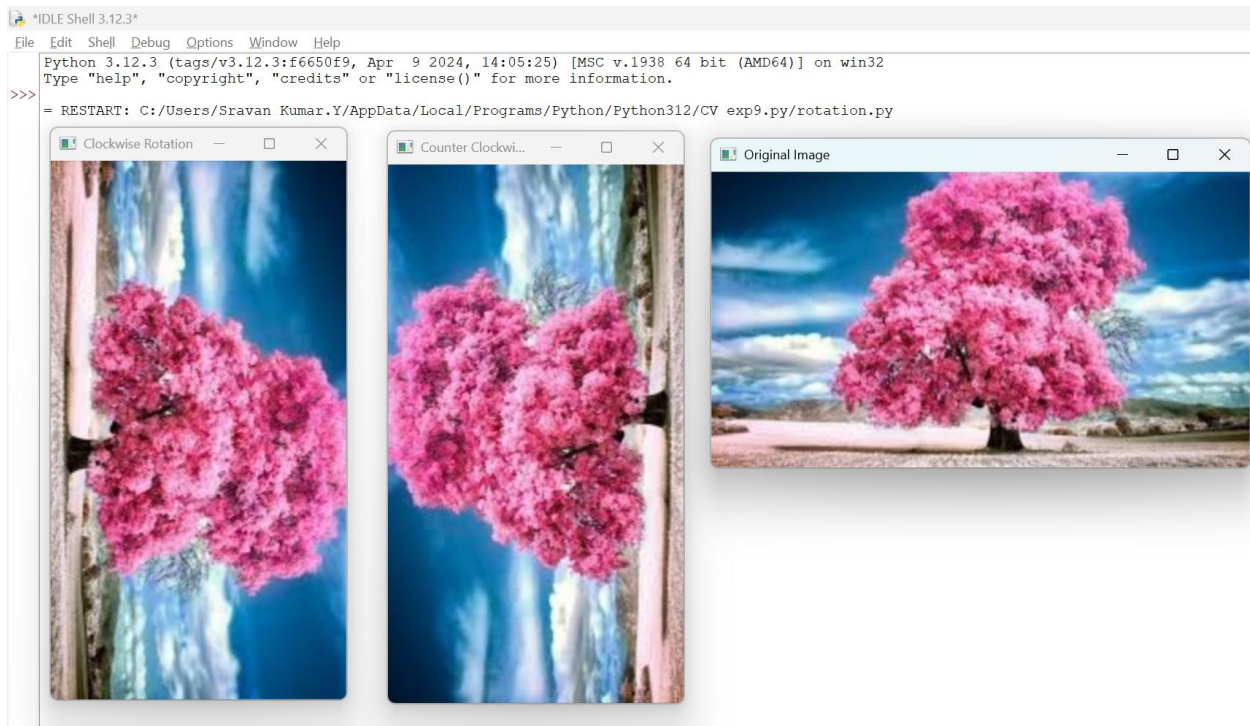
img = cv2.imread("image.jpg")

clockwise = cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE)
counter_clockwise = cv2.rotate(img, cv2.ROTATE_90_COUNTERCLOCKWISE)

cv2.imshow("Original Image", img)
cv2.imshow("Clockwise Rotation", clockwise)
cv2.imshow("Counter Clockwise Rotation", counter_clockwise)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output:



Experiment No. 10: Image Translation (Moving an Image from One Place to Another)

Aim

To read an image in Python and move (translate) it from one position to another using OpenCV.

Software Requirements

- Python 3.x
- OpenCV (cv2)
- NumPy
- Visual Studio Code (VS Code)

Procedure

1. Import the OpenCV and NumPy libraries.
2. Read the input image using cv2.imread().
3. Create a translation matrix.
4. Apply the translation using cv2.warpAffine().
5. Display the original and translated images.

Code

```
moving.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Pyt
File Edit Format Run Options Window Help
import cv2
import numpy as np

img = cv2.imread("image.jpg")
rows, cols = img.shape[:2]

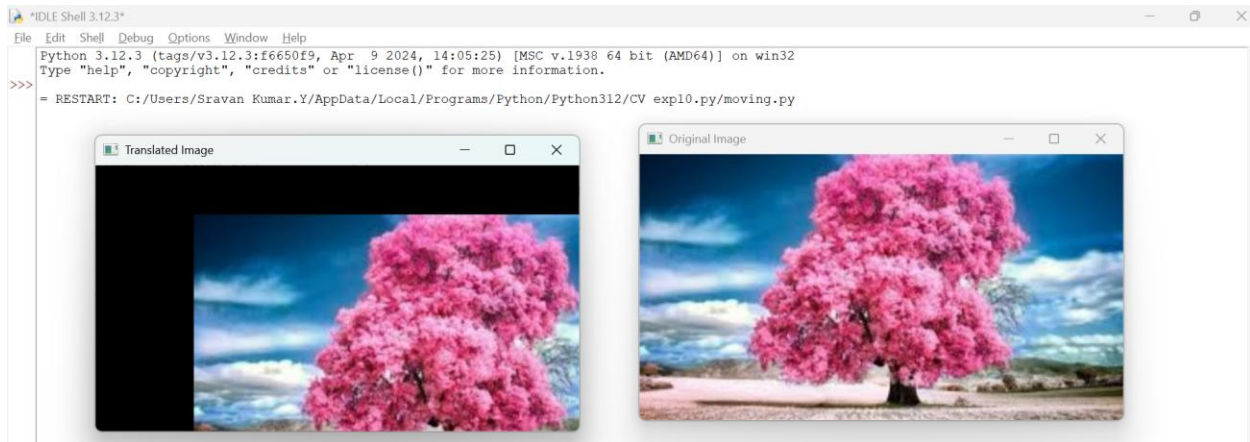
# Move image 100 pixels right and 50 pixels down
M = np.float32([[1, 0, 100],
                [0, 1, 50]])

translated = cv2.warpAffine(img, M, (cols, rows))

cv2.imshow("Original Image", img)
cv2.imshow("Translated Image", translated)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output



Result

The input image was successfully moved from one position to another using the image translation technique in OpenCV.

Experiment No. 11: Affine Transformation of an Image

Aim

To perform Affine Transformation on an image using OpenCV.

Software Requirements

- Python 3.x
- OpenCV (cv2)
- NumPy
- Visual Studio Code (VS Code)

Procedure

1. Import OpenCV and NumPy.
2. Read the input image.
3. Select three points from the original image.
4. Define three corresponding points for the transformed image.
5. Calculate the affine transformation matrix using `cv2.getAffineTransform()`.
6. Apply the transformation using `cv2.warpAffine()`.

7. Display the original and transformed images.

Code

```
AT.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/CV exp 11.py/AT.py (3.12.3)
File Edit Format Run Options Window Help
import cv2
import numpy as np

img = cv2.imread("image.jpg")
rows, cols = img.shape[:2]

# Three points from the original image
pts1 = np.float32([
    [50, 50],
    [200, 50],
    [50, 200]
])

# Corresponding points for transformation
pts2 = np.float32([
    [50, 100],
    [200, 50],
    [100, 200]
])

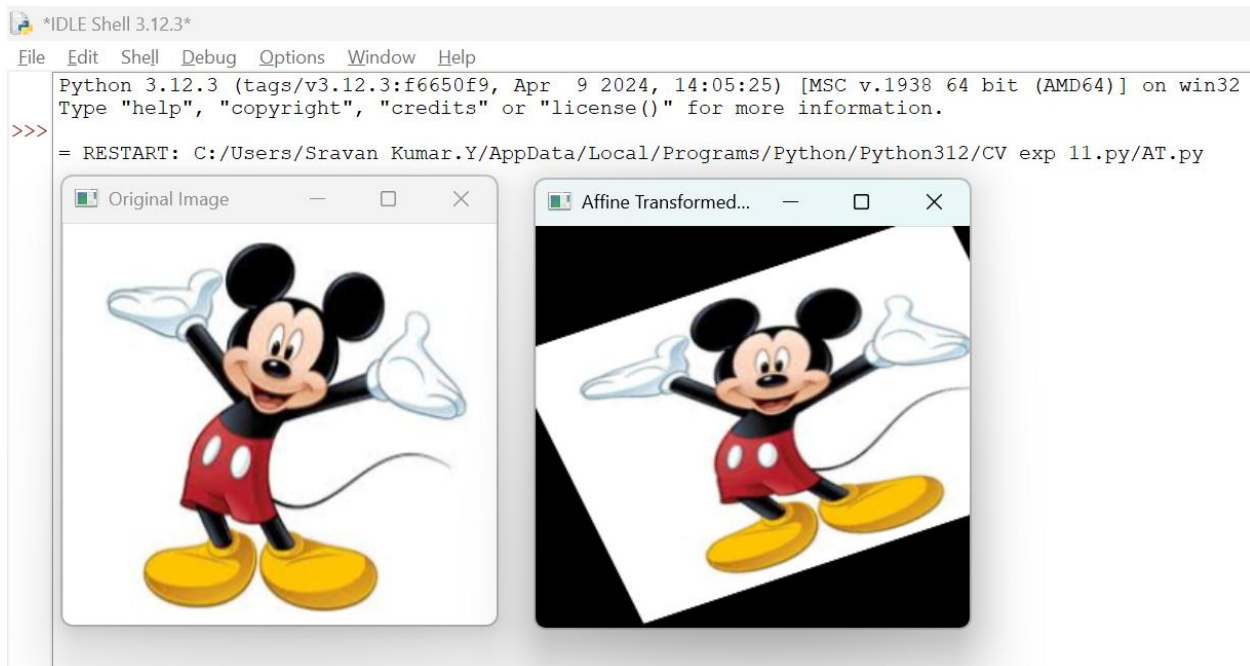
# Affine transformation matrix
M = cv2.getAffineTransform(pts1, pts2)

# Apply transformation
affine = cv2.warpAffine(img, M, (cols, rows))

cv2.imshow("Original Image", img)
cv2.imshow("Affine Transformed Image", affine)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output



Result

The image was successfully transformed using **Affine Transformation** in OpenCV.

Experiment No. 12: Perspective Transformation of an Image

Aim

Perspective Transformation on an image using OpenCV.

Software Requirements

- Python 3.x
- OpenCV (cv2)
- NumPy
- Visual Studio Code (VS Code)

Procedure

1. Import OpenCV and NumPy.
2. Read the input image.
3. Select four points from the original image.
4. Define four corresponding points for the transformed image.
5. Calculate the perspective transformation matrix using `cv2.getPerspectiveTransform()`.
6. Apply the transformation using `cv2.warpPerspective()`.
7. Display the original and transformed images.

Code

```

perspective.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python31
File Edit Format Run Options Window Help
import cv2
import numpy as np

img = cv2.imread("image.jpg")

# Four points from the original image
pts1 = np.float32([
    [50, 50],
    [400, 50],
    [50, 400],
    [400, 400]
])

# Corresponding points in the output image
pts2 = np.float32([
    [0, 0],
    [400, 0],
    [0, 400],
    [400, 400]
])

# Perspective transformation matrix
M = cv2.getPerspectiveTransform(pts1, pts2)

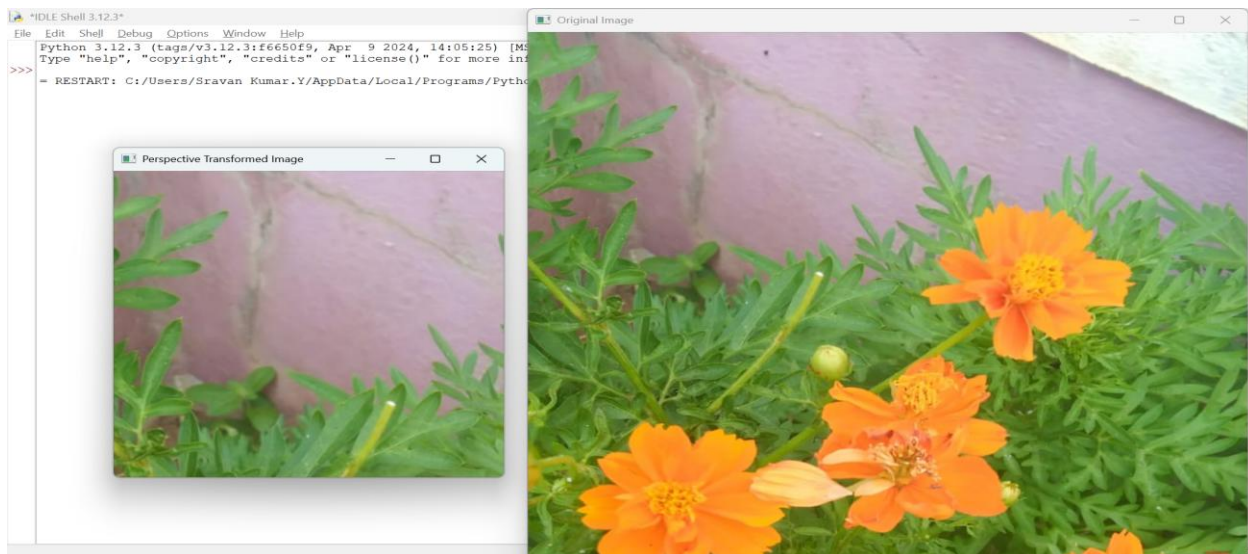
# Apply transformation
perspective = cv2.warpPerspective(img, M, (400, 400))

cv2.imshow("Original Image", img)
cv2.imshow("Perspective Transformed Image", perspective)

cv2.waitKey(0)
cv2.destroyAllWindows()

```

Output



Result

The image was successfully transformed using **Perspective Transformation** in OpenCV.

Experiment No. 13: Perspective Transformation on Video

Aim

Perspective Transformation on a video using OpenCV.

Software Requirements

- Python 3.x
- OpenCV (cv2)
- NumPy
- Visual Studio Code (VS Code)

Procedure

1. Import OpenCV and NumPy.
2. Read the video using cv2.VideoCapture().
3. Select four points from each video frame.
4. Create the perspective transformation matrix.
5. Apply cv2.warpPerspective() to each frame.
6. Display the original and transformed video.
7. Press 'q' to stop the video.

Code

```
perspective2.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/
File Edit Format Run Options Window Help
import cv2
import numpy as np

cap = cv2.VideoCapture("video.mp4")

while True:
    ret, frame = cap.read()

    if not ret:
        break

    pts1 = np.float32([
        [50, 50],
        [500, 50],
        [50, 400],
        [500, 400]
    ])

    pts2 = np.float32([
        [0, 0],
        [500, 0],
        [0, 400],
        [500, 400]
    ])

    M = cv2.getPerspectiveTransform(pts1, pts2)

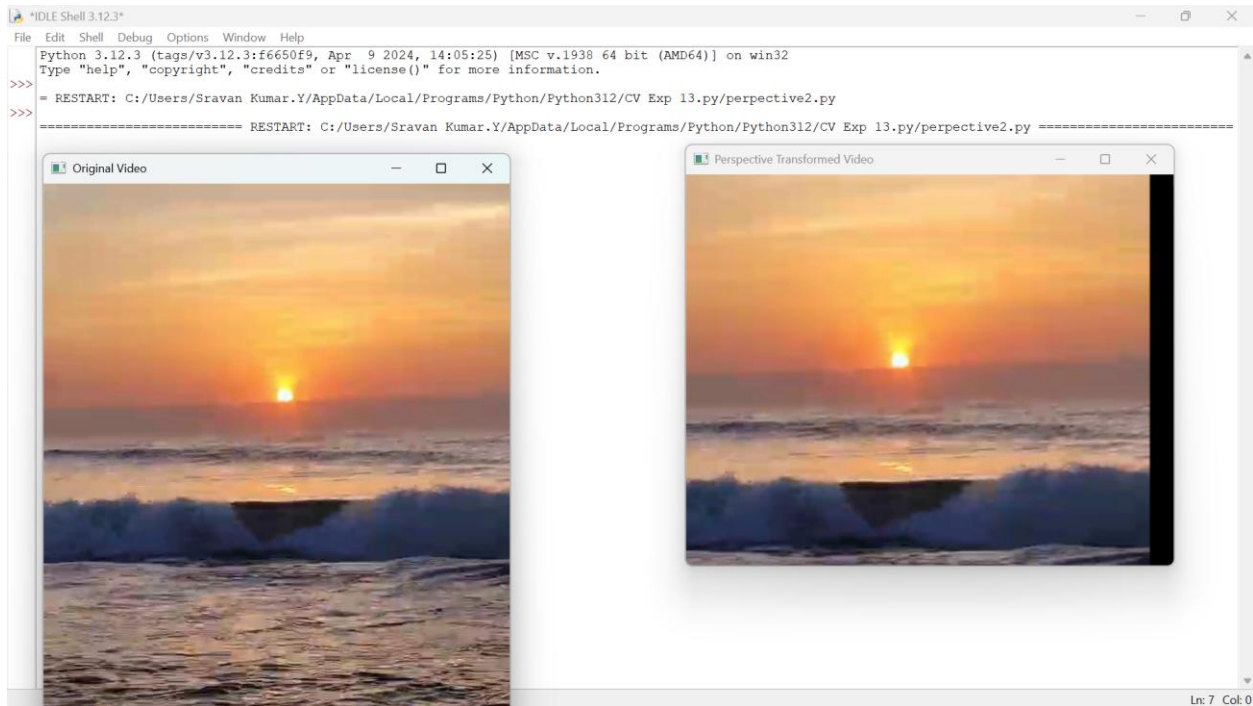
    transformed = cv2.warpPerspective(frame, M, (500, 400))

    cv2.imshow("Original Video", frame)
    cv2.imshow("Perspective Transformed Video", transformed)

    if cv2.waitKey(30) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Output



Result

The video was successfully processed, and perspective transformation was applied to each frame using OpenCV.

Experiment No. 14: Image Transformation using Homography Matrix

Aim

To perform image transformation using a Homography Matrix in OpenCV.

Procedure

1. Import OpenCV and NumPy.
2. Read the input image.
3. Select four points from the original image.
4. Define four corresponding destination points.
5. Calculate the homography matrix using `cv2.findHomography()`.
6. Apply the transformation using `cv2.warpPerspective()`.
7. Display the original and transformed images.

CODE

```
cv exp14.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/cv exp14.py/cv exp14.py (3.12.3)
File Edit Format Run Options Window Help

import cv2
import numpy as np

# Read the input image
image = cv2.imread("input.jpg")

# Get image dimensions
height, width = image.shape[:2]

# Define four points in the original image
src_points = np.float32([
    [100, 100],
    [400, 100],
    [400, 300],
    [100, 300]
])

# Define corresponding points in the output image
dst_points = np.float32([
    [50, 50],
    [450, 80],
    [400, 350],
    [80, 300]
])

# Calculate Homography Matrix
H, status = cv2.findHomography(src_points, dst_points)

print("Homography Matrix:")
print(H)

# Apply homography transformation
result = cv2.warpPerspective(image, H, (width, height))

# Display images
cv2.imshow("Original Image", image)
cv2.imshow("Homography Transformed Image", result)

cv2.waitKey(0)
cv2.destroyAllWindows()

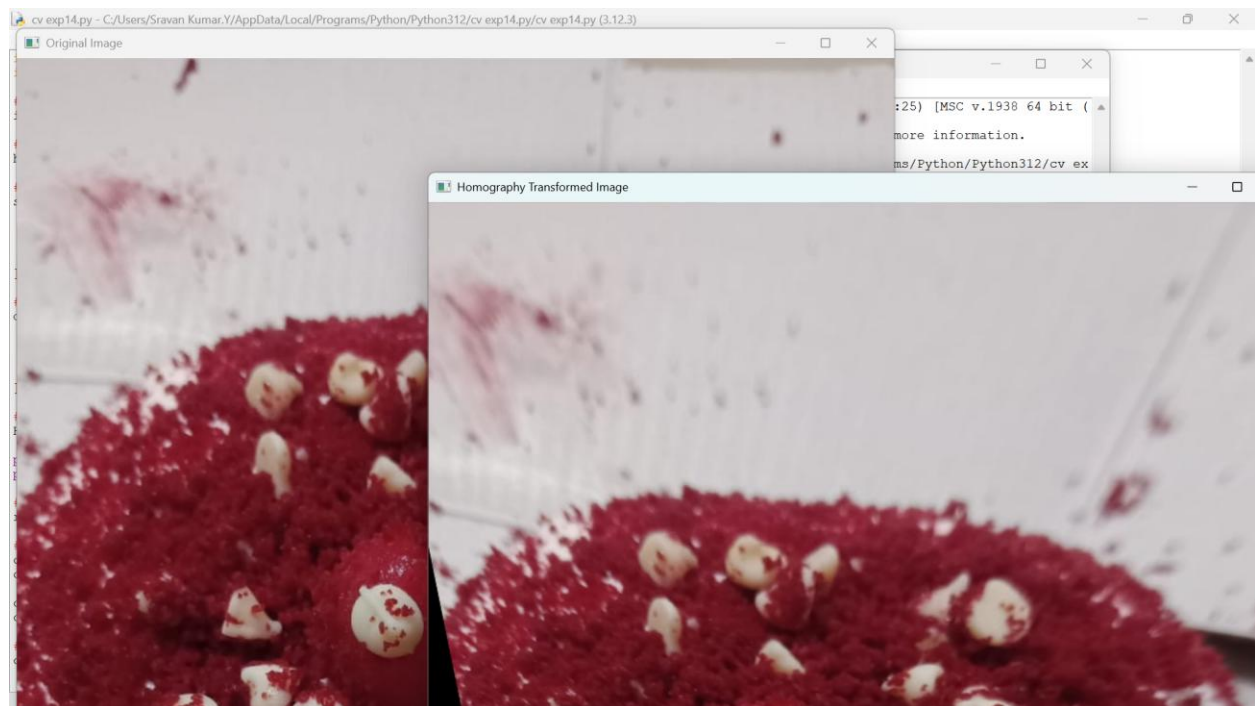
# Save the output
cv2.imwrite("output.jpg", result)
```

```
*IDLE Shell 3.12.3*
File Edit Shell Debug Options Window Help

Python 3.12.3 (tags/v3.12.3:f6650f9, Apr 9 2024, 14:05:25) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>>
= RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/cv exp14.py/cv exp14.py
Homography Matrix:
[[ 1.25426483e+00  2.62896019e-01 -9.75528026e+01]
 [ 7.45329001e-02  1.73060520e+00 -1.26350528e+02]
 [-4.22420796e-04  1.25507717e-03  1.00000000e+00]]
```

Ln: 1 Col: 9



Result

Thus, the input image was successfully transformed using a Homography Matrix, and the perspective-transformed image was obtained.

ExperimentNo.15 :Perform Transformation Using Direct Linear Transformation (DLT)

Aim

To perform image transformation using the **Direct Linear Transformation (DLT)** method and obtain the transformed image.

Procedure

1. Import the required OpenCV and NumPy libraries.
2. Read the input image.
3. Select four corresponding points from the source and destination images.
4. Construct the DLT equations using the corresponding points.
5. Solve the equations using Singular Value Decomposition (SVD).
6. Obtain the **3×3 transformation matrix**.
7. Apply the transformation using `cv2.warpPerspective()`.
8. Display and save the transformed image.

Code

```
exp 15 dlt.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/cv exp 15.py/exp 15 dlt.py (3.12.3)
File Edit Format Run Options Window Help

# Normalize Homography Matrix
H = H / H[2, 2]

# -----
# 7. Display DLT Transformation Matrix
# -----
print("\nDLT Transformation Matrix:")
print(H)

# -----
# 8. Get image size
# -----
height, width = image.shape[:2]

# -----
# 9. Apply DLT transformation
# -----
result = cv2.warpPerspective(
    image,
    H,
    (width, height)
)

# -----
# 10. Display images
# -----
cv2.imshow("Original Image", image)
cv2.imshow("DLT Transformed Image", result)

# -----
# 11. Save output
# -----
output_path = os.path.join(folder, "dlt_output.jpg")
cv2.imwrite(output_path, result)
print("\nOutput saved as:")
print(output_path)

# -----
# 12. Close windows
# -----
cv2.waitKey(0)
cv2.destroyAllWindows()
```

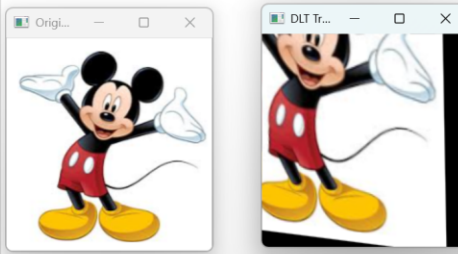
```
*IDLE Shell 3.12.3*
File Edit Shell Debug Options Window Help

Python 3.12.3 (tags/v3.12.3:f6650f9, Apr 9 2024, 14:05:25) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>>
= RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/cv exp 15.py/exp 15 dlt.py
Image loaded successfully!

DLT Transformation Matrix:
[[ 1.25426483e+00  2.62896019e-01 -9.75528026e+01]
 [ 7.45329001e-02  1.73060520e+00 -1.26350528e+02]
 [-4.22420796e-04  1.25507717e-03  1.00000000e+00]]

Output saved as:
C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/cv exp 15.py/dlt_output.jpg
```



Result

Thus, the image transformation was successfully performed using the Direct Linear Transformation (DLT) method

Experiment No.16: Perform Edge Detection Using Canny Method

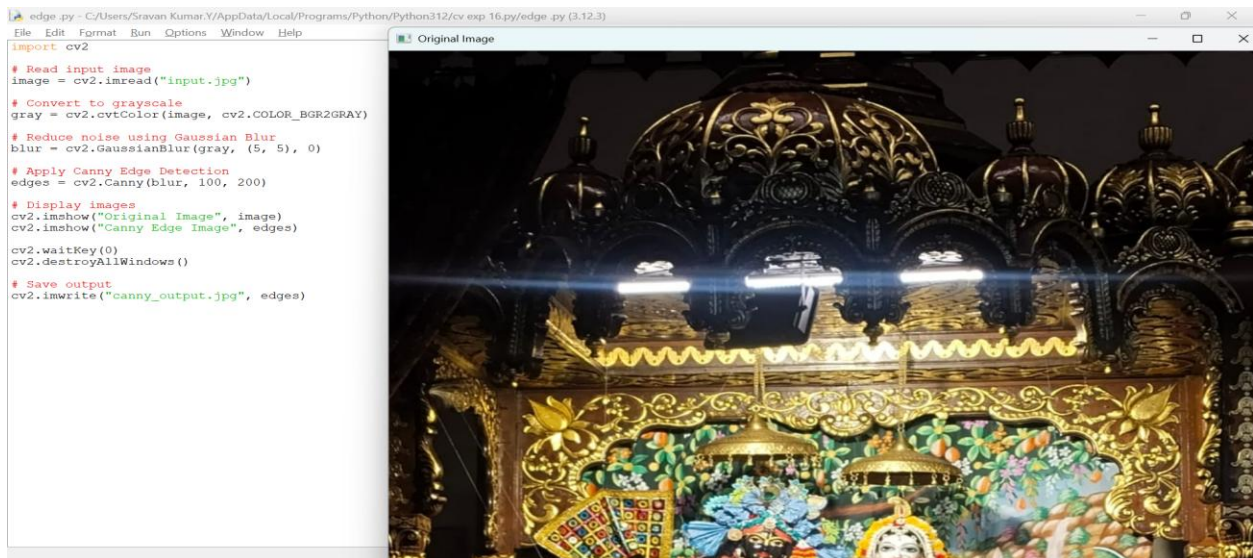
Aim

- To detect the edges of an image using the Canny Edge Detection method.

Procedure

1. Import OpenCV and NumPy.
2. Read the input image.
3. Convert the image from RGB/BGR to grayscale.
4. Apply Gaussian Blur to reduce noise.
5. Apply the Canny edge detection algorithm.
6. Display the original and detected edge images.
7. Save the resulting edge image.

Code



Output



Result

Thus, the edges of the input image were successfully detected using the Canny edge detection method.

Experiment No.17: Perform Edge Detection Using Sobel Matrix Along X-Axis

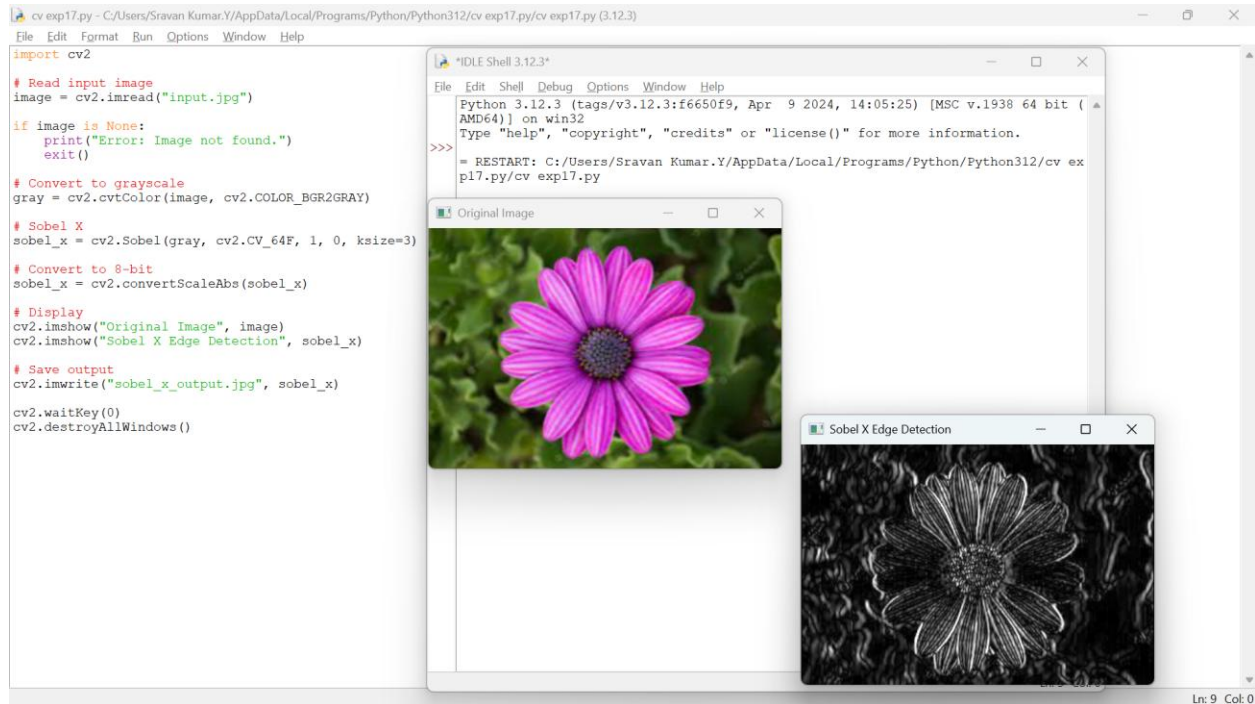
Aim

To detect vertical edges in an image using the **Sobel operator along the X-axis**.

Procedure

1. Read the input image.
2. Convert the image into grayscale.
3. Apply the Sobel operator in the X direction.
4. Convert the result into an absolute 8-bit image.
5. Display and save the detected edges.

Code



Result

Thus, vertical edges in the input image were successfully detected using the Sobel operator along with the X-axis.

Experiment No.18: Perform Edge Detection Using Sobel Matrix Along Y-Axis

Aim

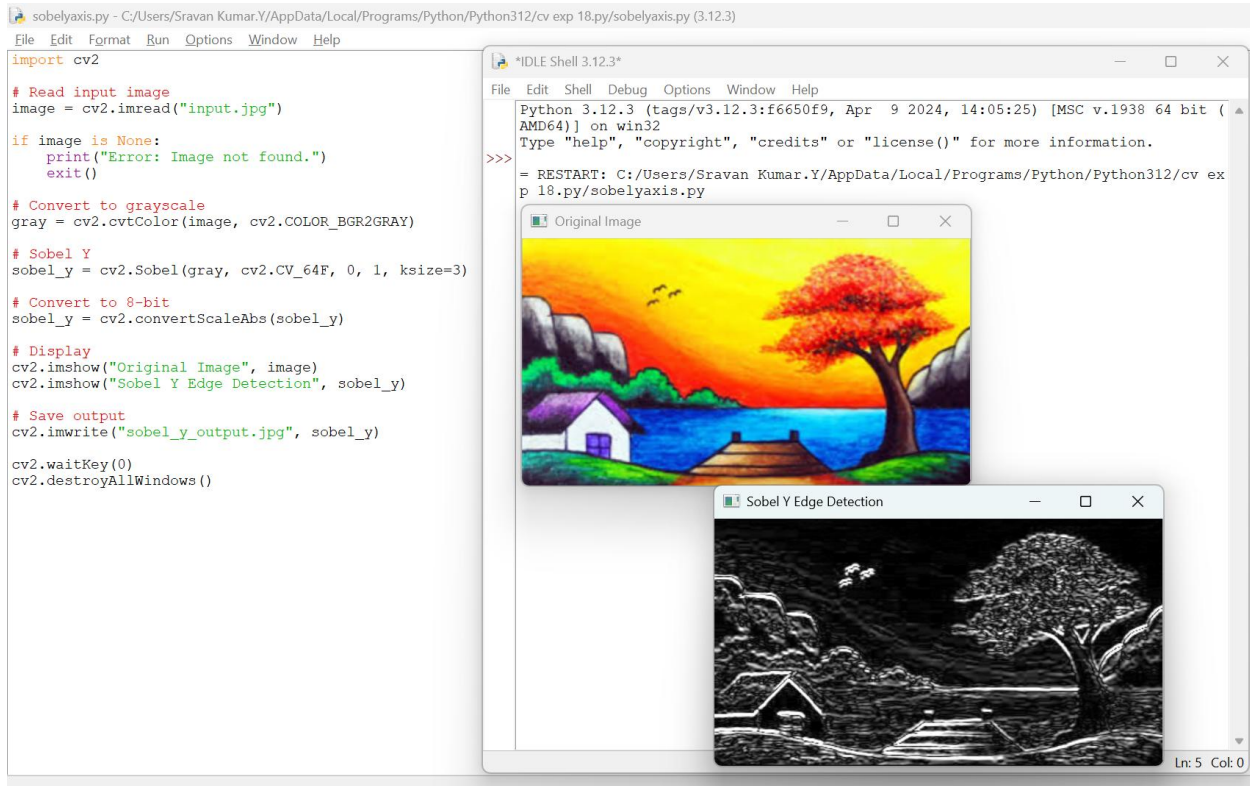
To detect horizontal edges in an image using the **Sobel operator along the Y-axis**.

Procedure

1. Read the input image.
2. Convert it to grayscale.
3. Apply the Sobel operator in the Y direction.
4. Convert the result to an 8-bit image.

5. Display and save the detected edges.

Code



Result

Thus, horizontal edges in the input image were successfully detected using the Sobel operator along the Y-axis.

Experiment No.19: Perform Edge Detection Using Sobel Matrix Along XY-Axis

Aim

To detect edges in both the **X and Y directions** using the Sobel operator.

Procedure

1. Read the input image.
2. Convert it to grayscale.

3. Calculate the Sobel X gradient.
4. Calculate the Sobel Y gradient.
5. Combine the X and Y gradients.
6. Display and save the final edge image.

Code

```
XY axis exp19.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/cv exp19.py/XY axis exp19.py (3.12.3)
File Edit Format Run Options Window Help

# Convert to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Sobel X
sobel_x = cv2.Sobel(
    gray,
    cv2.CV_64F,
    1,
    0,
    ksize=3
)

# Sobel Y
sobel_y = cv2.Sobel(
    gray,
    cv2.CV_64F,
    0,
    1,
    ksize=3
)

# Convert to absolute values
sobel_x = cv2.convertScaleAbs(sobel_x)
sobel_y = cv2.convertScaleAbs(sobel_y)

# Combine X and Y
sobel_xy = cv2.addWeighted(
    sobel_x, 0.5,
    sobel_y, 0.5,
    0
)

# Display
cv2.imshow("Original Image", image)
cv2.imshow("Sobel X", sobel_x)
cv2.imshow("Sobel Y", sobel_y)
cv2.imshow("Sobel XY Edge Detection", sobel_xy)

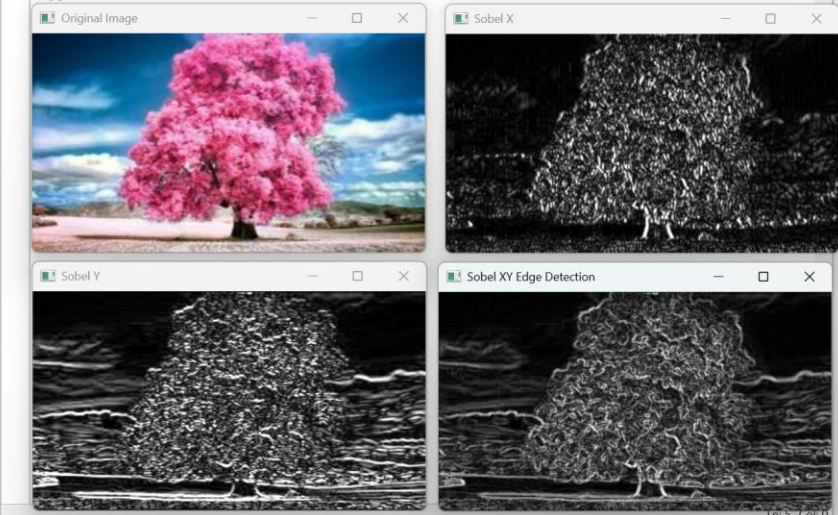
# Save output
cv2.imwrite("sobel_xy_output.jpg", sobel_xy)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

IDLE Shell 3.12.3

Python 3.12.3 (tags/v3.12.3:f6650f9, Apr 9 2024, 14:05:25) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>> = RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/cv exp19.py/XY axis exp19.py



Original Image

Sobel X

Sobel Y

Sobel XY Edge Detection

Ln: 10 Col: 0

Result

Thus, edges in both X and Y directions were successfully detected using the Sobel operator.

Experiment No. 20

Perform Sharpening of Image Using Laplacian Mask with Negative Center Coefficient

Aim:

To perform image sharpening using a Laplacian mask with a negative center coefficient.

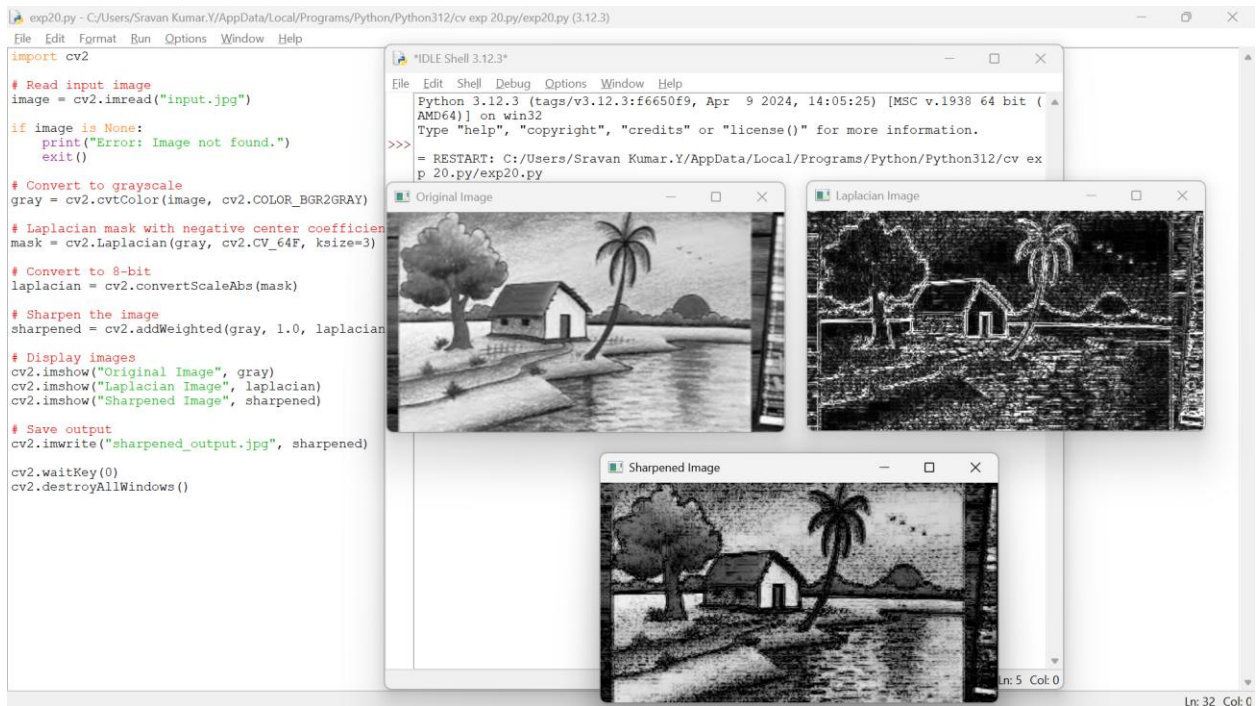
Laplacian Mask:

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Procedure:

1. Read the input image.
2. Convert the image into grayscale.
3. Apply the Laplacian mask to the image.
4. Subtract the Laplacian image from the original image.
5. Display the original and sharpened images.
6. Save the sharpened image.

Code:



Result:

Thus, the image was successfully sharpened using a **Laplacian mask with a negative center coefficient**.

Experiment 21

Perform Sharpening of Image Using Laplacian Mask with Diagonal Neighbors

Aim

To sharpen an image using a **Laplacian mask that includes diagonal neighbors**.

Laplacian Mask

1	1	1
1	-8	1
1	1	1

This mask considers **eight neighboring pixels**.

Procedure

1. Read the input image.
2. Convert it to grayscale.
3. Apply the 8-neighbor Laplacian mask.
4. Subtract the Laplacian component from the original image.
5. Display and save the sharpened image.

Code

```
exp21.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/cv exp21.py/exp21.py (3.12.3)
File Edit Format Run Options Window Help

import cv2
import numpy as np

# Read input image
image = cv2.imread("input.jpg")

if image is None:
    print("Error: Image not found.")
    exit()

# Convert to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# 8-neighbor Laplacian mask
mask = np.array([
    [1, 1, 1],
    [1, -8, 1],
    [1, 1, 1]
], dtype=np.float32)

# Apply Laplacian mask
laplacian = cv2.filter2D(
    gray,
    cv2.CV_32F,
    mask
)

# Sharpen image
sharpened = gray.astype(np.float32) - laplacian

# Clip values to valid range
sharpened = np.clip(sharpened, 0, 255).astype(np.uint8)

# Display
cv2.imshow("Original Image", gray)
cv2.imshow("Sharpened Image", sharpened)

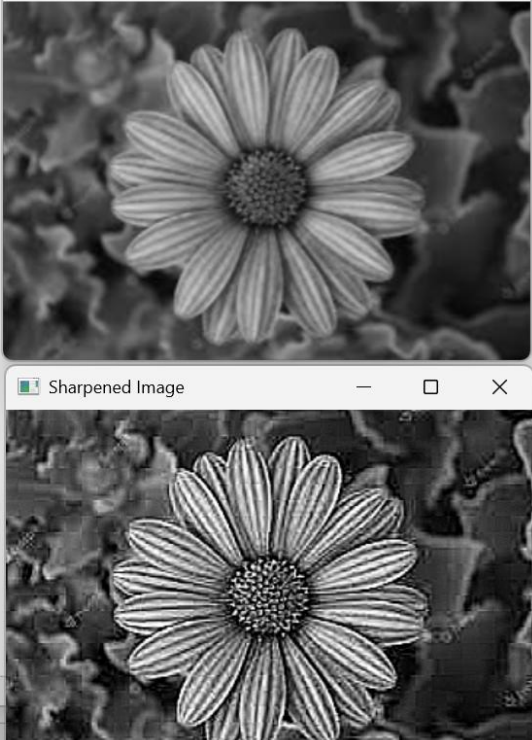
# Save output
cv2.imwrite("sharpened_diagonal.jpg", sharpened)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

```
*IDLE Shell 3.12.3*
File Edit Shell Debug Options Window Help

Python 3.12.3 (tags/v3.12.3:f6650f9, Apr 9 20:
AMD64)] on win32
Type "help", "copyright", "credits" or "licens
>>>
= RESTART: C:/Users/Sravan Kumar.Y/AppData/Loc
p21.py/exp21.py

Original Image
Sharpened Image
```



Result

Thus, the image was successfully sharpened using an 8-neighbor Laplacian mask with diagonal neighbors.

Experiment 22: Image Sharpening Using Laplacian Mask with Positive Center Coefficient

Aim

To perform **image sharpening using a Laplacian mask with a positive center coefficient** using OpenCV.

Principle

The Laplacian sharpening mask used is:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

The sharpened image is obtained by applying this mask to the original image.

Formula:

$$g(x,y) = 5f(x,y) - [f(x+1,y) + f(x-1,y) + f(x,y+1) + f(x,y-1)]$$

Procedure

1. Import the OpenCV library.
2. Read the input image.
3. Define the Laplacian sharpening mask.
4. Apply convolution using `cv2.filter2D()`.
5. Generate the sharpened image.
6. Display the original and sharpened images.
7. Save the sharpened image.
8. Close all windows.

Code

```
exp22.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/cv exp22.py/exp22.py (3.12.3)
File Edit Format Run Options Window Help

import cv2
import numpy as np

# Read input image
image = cv2.imread("input.jpg")

if image is None:
    print("Error: Image not found")
    exit()

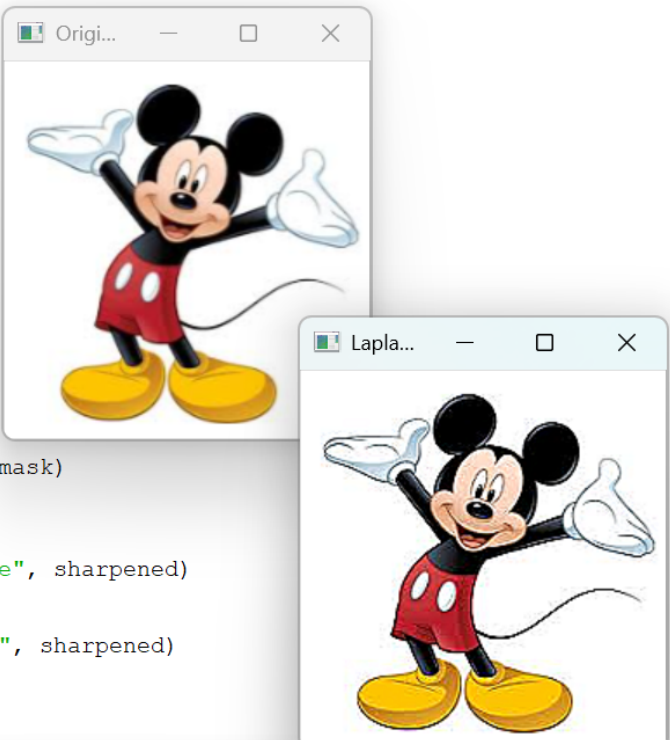
# Laplacian sharpening mask
mask = np.array([
    [0, -1, 0],
    [-1, 5, -1],
    [0, -1, 0]
], dtype=np.float32)

# Apply convolution mask
sharpened = cv2.filter2D(image, -1, mask)

# Display images
cv2.imshow("Original Image", image)
cv2.imshow("Laplacian Sharpened Image", sharpened)

# Save output
cv2.imwrite("sharpened_laplacian.jpg", sharpened)

cv2.waitKey(0)
cv2.destroyAllWindows()
```



```
*IDLE Shell 3.12.3*
File Edit Shell Debug Options Window Help

Python 3.12.3 (tags/v3.12.3:f6650f9, Apr 9 2024, 14:05:25) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/cv exp22.py/exp22.py
```

Result

Thus, the input image was successfully sharpened using a **Laplacian mask with a positive center coefficient**.

Experiment 23: Image Sharpening Using Unsharp Masking

Aim

To perform **image sharpening using unsharp masking** using OpenCV.

Principle

Unsharp masking sharpens an image by subtracting a blurred version of the image from the original image and combining the result appropriately.

Basic concept:

Sharpened Image = Original Image + (Original Image - Blurred Image)

The term:

Original Image - Blurred Image

represents the high-frequency details of the image.

Procedure

1. Import OpenCV.
2. Read the input image.
3. Create a blurred version using Gaussian blur.
4. Calculate the difference between the original and blurred images.
5. Add the high-frequency details to the original image.
6. Display the original and sharpened images.
7. Save the sharpened image.

Code

```
exp23.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/cv exp23.py/exp23.py (3.12.3)
File Edit Format Run Options Window Help

import cv2

# Read input image
image = cv2.imread("input.jpg")

if image is None:
    print("Error: Image not found")
    exit()

# Create blurred image
blurred = cv2.GaussianBlur(image, (5, 5), 0)

# Unsharp masking
sharpened = cv2.addWeighted(
    image, 1.5,
    blurred, -0.5,
    0
)

# Display images
cv2.imshow("Original Image", image)
cv2.imshow("Blurred Image", blurred)
cv2.imshow("Unsharp Masked Image", sharpened)

# Save output
cv2.imwrite("unsharp_output.jpg", sharpened)

cv2.waitKey(0)
cv2.destroyAllWindows()

*IDLE Shell 3.12.3*
File Edit Shell Debug Options Window Help
Python 3.12.3 (tags/v3.12.3:f6650f9, Apr 9 2024, 14:05:25) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/cv exp23.py/exp23.py
```

Result

Thus, image sharpening was successfully performed using the **unsharp masking technique**.

Experiment 24: Image Sharpening Using High-Boost Mask

Aim

To perform **image sharpening using a high-boost mask** using OpenCV.

Principle

A high-boost filter enhances high-frequency components while retaining the original image information.

The high-boost mask is based on the parameter **A**.

A general high-boost mask can be represented as:

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & A+8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

For the 4-neighbor form:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & A+4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

When:

$$A = 1$$

the mask becomes a standard Laplacian sharpening mask:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Procedure

1. Import OpenCV.
2. Read the input image.
3. Set the high-boost coefficient A.
4. Create the high-boost mask.
5. Apply convolution using `filter2D()`.
6. Display the original and sharpened images.
7. Save the result.

Code

```
exp24.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/cv exp24.py/exp24.py (3.12.3)
File Edit Format Run Options Window Help

import cv2
import numpy as np

# Read input image
image = cv2.imread("input.jpg")

if image is None:
    print("Error: Image not found")
    exit()

# High-boost coefficient
A = 2

# High-boost mask
mask = np.array([
    [0, -1, 0],
    [-1, A + 4, -1],
    [0, -1, 0]
], dtype=np.float32)

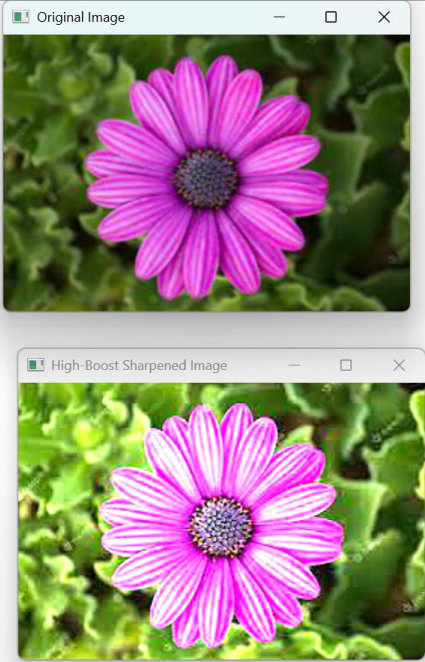
# Apply high-boost mask
sharpened = cv2.filter2D(image, -1, mask)

# Display
cv2.imshow("Original Image", image)
cv2.imshow("High-Boost Sharpened Image", sharpened)

# Save output
cv2.imwrite("highboost_output.jpg", sharpened)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

```
*IDLE Shell 3.12.3*
File Edit Shell Debug Options Window Help
Python 3.12.3 (tags/v3.12.3:f6650f9, Apr 9 2024, 14:05:25) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> = RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/cv exp24.py/exp24.py
```



Result

Thus, the image was successfully sharpened using a **high-boost mask**. The value of A controls the amount of sharpening.

Experiment 25: Image Sharpening Using Gradient Masking

Aim

To perform **image sharpening using gradient masks** using OpenCV.

Principle

Gradient masks detect changes in image intensity and enhance edges.

Common gradient operators include:

Sobel X Mask

-1	0	1
-2	0	2
-1	0	1

Sobel Y Mask

-1	-2	-1
0	0	0
1	2	1

The gradient magnitude is calculated from the horizontal and vertical gradients.

Procedure

1. Import OpenCV.
2. Read the input image.
3. Convert it to grayscale.
4. Apply Sobel X filtering.
5. Apply Sobel Y filtering.
6. Calculate the gradient magnitude.
7. Normalize the result.
8. Display and save the gradient image.

Code

```
exp25.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/cv exp25.py/exp25.py (3.12.3)
File Edit Format Run Options Window Help

import cv2
import numpy as np

# Read image
image = cv2.imread("input.jpg")

if image is None:
    print("Error: Image not found")
    exit()

# Convert to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Sobel X
gx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)

# Sobel Y
gy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

# Calculate gradient magnitude
magnitude = cv2.magnitude(
    gx.astype(np.float32),
    gy.astype(np.float32)
)

# Convert to 8-bit image
magnitude = cv2.normalize(
    magnitude,
    None,
    0,
    255,
    cv2.NORM_MINMAX
).astype(np.uint8)

# Display
cv2.imshow("Original Image", image)
cv2.imshow("Gradient Masking", magnitude)

# Save output
cv2.imwrite("gradient_output.jpg", magnitude)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

```
*IDLE Shell 3.12.3*
File Edit Shell Debug Options Window Help

Python 3.12.3 (tags/v3.12.3:f6650f9, Apr 9 2024, 14:05:25) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/cv exp25.py/exp25.py

Original Image
Gradient Masking

Ln: 5 Col: 0
```

Result

Thus, the important edges and intensity changes in the image were successfully detected using **gradient masking**.

Experiment 26: Insert Watermark into an Image Using OpenCV

Aim

To insert a **text watermark** into an image using OpenCV.

Procedure

1. Import OpenCV.
2. Read the input image.
3. Define the watermark text.
4. Select the position, font, size, and color.
5. Add the watermark using `cv2.putText()`.
6. Display the watermarked image.
7. Save the output image.

Code

```
exp26.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/cv exp26.py/exp26.py (3.12.3)
File Edit Format Run Options Window Help

import cv2

# Read input image
image = cv2.imread("input.jpg")

if image is None:
    print("Error: Image not found")
    exit()

# Watermark text
text = "SIMATS"

# Position
position = (30, 50)

# Font
font = cv2.FONT_HERSHEY_SIMPLEX

# Add watermark
cv2.putText(
    image,
    text,
    position,
    font,
    1,
    (255, 255, 255),
    2,
    cv2.LINE_AA
)

# Display
cv2.imshow("Watermarked Image", image)

# Save
cv2.imwrite("watermarked_output.jpg", image)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

```
*IDLE Shell 3.12.3*
File Edit Shell Debug Options Window Help
Python 3.12.3 (tags/v3.12.3:f6650f9, Apr 9 2024, 14:05:25) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/cv exp26.py/exp26.py
```



Result

Thus, a watermark was successfully inserted into the input image using OpenCV.

Experiment 27: Cropping, Copying and Pasting Image Inside Another Image

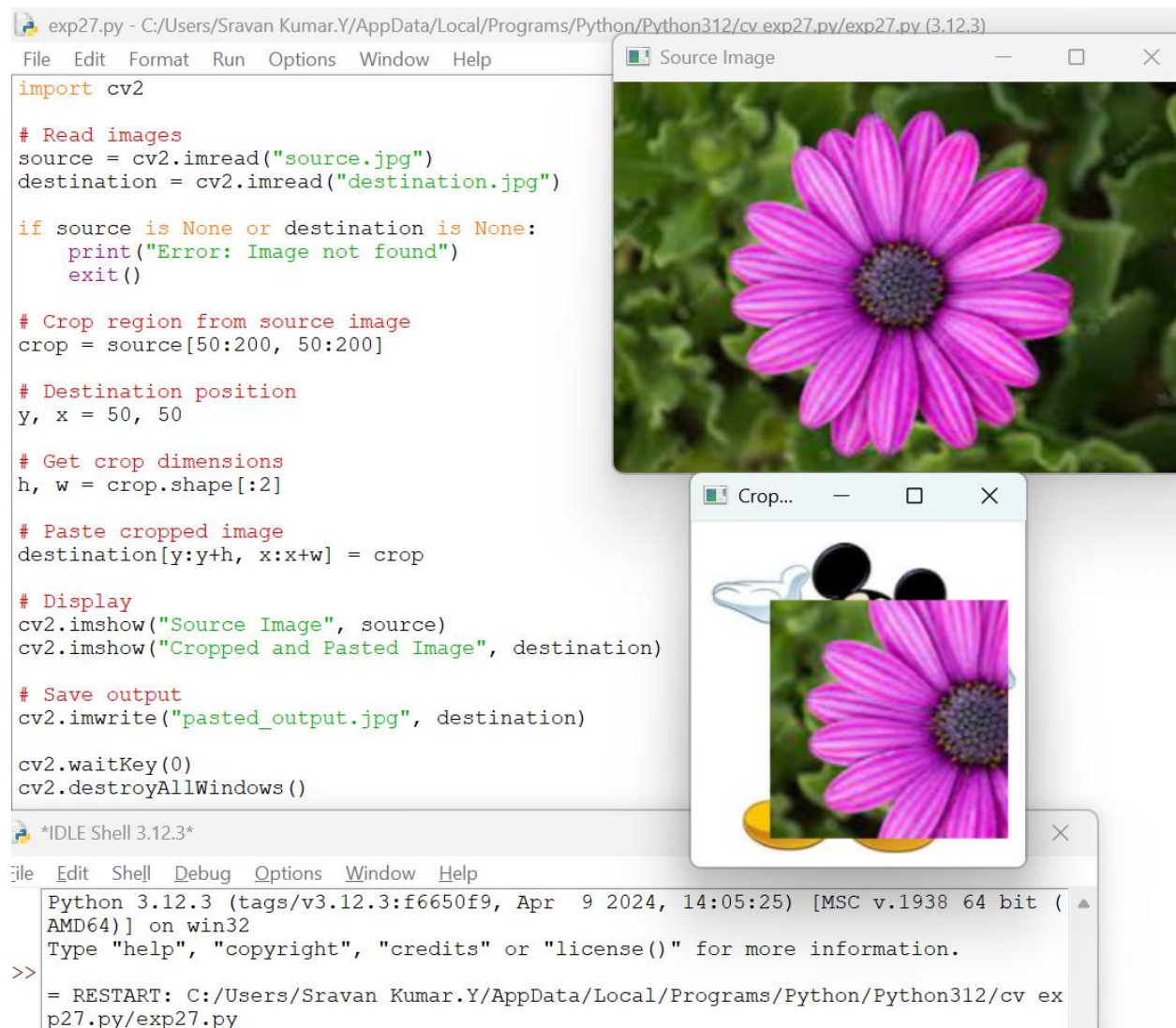
Aim

To perform **cropping, copying, and pasting of an image region inside another image** using OpenCV.

Procedure

1. Import OpenCV.
2. Read the source image.
3. Read the destination image.
4. Select a rectangular region from the source image.
5. Copy the selected region.
6. Select the destination location.
7. Paste the cropped region into the destination image.
8. Display and save the result.

Code



Result

Thus, cropping, copying, and pasting of an image region into another image was successfully performed using OpenCV.

Experiment 28: Find the Boundary of an Image Using a Convolution Kernel

Aim

To find the **boundary or edges of an image using a convolution kernel** in OpenCV.

Principle

A convolution kernel is applied to the image to detect areas where there is a significant change in intensity.

A simple edge-detection kernel is:

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

The center coefficient emphasizes the current pixel, while neighboring negative coefficients detect intensity changes.

Procedure

1. Import OpenCV.
2. Read the input image.
3. Convert the image to grayscale.
4. Define the convolution kernel.
5. Apply the kernel using `cv2.filter2D()`.
6. Display the original and boundary images.
7. Save the boundary image.
8. Close all windows.

Code

```
exp28.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/cv exp28.py/exp28.py (3.12.3)
File Edit Format Run Options Window Help

import cv2
import numpy as np

# Read input image
image = cv2.imread("input.jpg")

if image is None:
    print("Error: Image not found")
    exit()

# Convert to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Boundary detection kernel
kernel = np.array([
    [-1, -1, -1],
    [-1,  8, -1],
    [-1, -1, -1]
], dtype=np.float32)

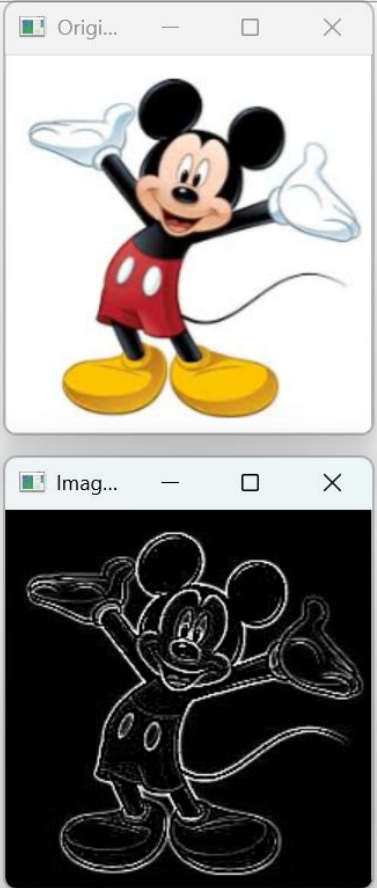
# Apply convolution
boundary = cv2.filter2D(gray, -1, kernel)

# Convert absolute values for display
boundary = cv2.convertScaleAbs(boundary)

# Display
cv2.imshow("Original Image", image)
cv2.imshow("Image Boundary", boundary)

# Save output
cv2.imwrite("boundary_output.jpg", boundary)

cv2.waitKey(0)
cv2.destroyAllWindows()
```



```
*IDLE Shell 3.12.3*
File Edit Shell Debug Options Window Help
Python 3.12.3 (tags/v3.12.3:f6650f9, Apr 9 2024, 14:05:25) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/cv exp28.py/exp28.py
```

Result

Thus, the boundaries of objects in the input image were successfully detected using a **convolution kernel in OpenCV**.

Experiment 29 – Morphological Operations

Using Erosion Technique

Aim

To perform morphological **erosion** on an image using OpenCV.

Procedure

1. Import the OpenCV library.
2. Read the input image.
3. Create a morphological structuring element (kernel).
4. Apply the erosion operation using `cv2.erode()`.
5. Display the original and eroded images.
6. Save the resulting image.

Code

```
exp29.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/cv exp29.py/exp29.py (3.12.3)
File Edit Format Run Options Window Help
import cv2

# Read input image
image = cv2.imread("input.jpg")

# Create kernel
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))

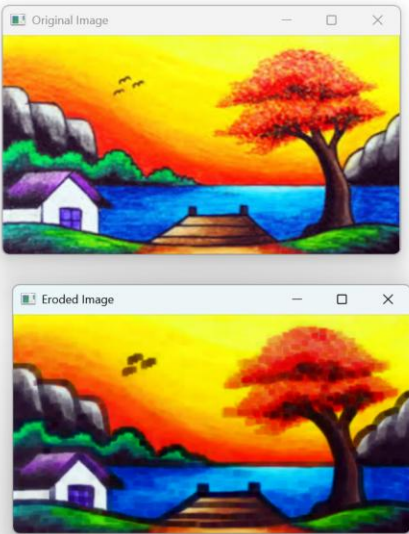
# Apply erosion
eroded = cv2.erode(image, kernel, iterations=1)

# Display images
cv2.imshow("Original Image", image)
cv2.imshow("Eroded Image", eroded)

# Save output
cv2.imwrite("erosion_output.jpg", eroded)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

```
*IDLE Shell 3.12.3*
File Edit Shell Debug Options Window Help
Python 3.12.3 (tags/v3.12.3:f6650f9, Apr 9 2024, 14:05:25) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> - RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/cv exp29.py/exp29.py
```



Result

Thus, the **erosion operation** was successfully performed using OpenCV.

Experiment 30 – Morphological Operations Using Dilation Technique

Aim

To perform morphological **dilation** on an image using OpenCV.

Procedure

1. Import the OpenCV library.
2. Read the input image.
3. Create a structuring element.
4. Apply dilation using `cv2.dilate()`.
5. Display the original and dilated images.
6. Save the output image.

Code

```
exp30.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/cv exp30.py/exp30.py (3.12.3)
File Edit Format Run Options Window Help

import cv2

# Read input image
image = cv2.imread("input.jpg")

if image is None:
    print("Error: Image not found.")
    exit()

# Create kernel
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))

# Apply dilation
dilated = cv2.dilate(image, kernel, iterations=1)

# Display images
cv2.imshow("Original Image", image)
cv2.imshow("Dilated Image", dilated)

# Save output
cv2.imwrite("dilation_output.jpg", dilated)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

The screenshot shows the IDLE Shell interface with the Python code for image dilation. The code imports cv2, reads 'input.jpg', creates a 5x5 rectangular kernel, applies a single iteration of dilation, and displays both the original and dilated images. The output shows two windows: 'Original Image' and 'Dilated Image'. Both windows display a colorful landscape painting of a house, a tree, and a body of water under a sunset sky. The 'Dilated Image' window shows the same image but with slightly expanded edges, which is not clearly visible in this low-resolution screenshot.

Result

Thus, the **dilation operation** was successfully performed using OpenCV.

Experiment 31 – Morphological Operations Using Opening Technique

Aim

To perform morphological **opening** on an image using OpenCV.

Procedure

1. Import OpenCV.
2. Read the input image.
3. Create a structuring element.
4. Apply the opening operation using `cv2.morphologyEx()`.
5. Display the original and opening images.
6. Save the output image.

Code

```
exp31.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/cv exp31.py/exp31.py (3.12.3)
File Edit Format Run Options Window Help

import cv2

# Read input image
image = cv2.imread("input.jpg")

if image is None:
    print("Error: Image not found.")
    exit()

# Create kernel
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))

# Apply opening
opening = cv2.morphologyEx(
    image,
    cv2.MORPH_OPEN,
    kernel
)

# Display images
cv2.imshow("Original Image", image)
cv2.imshow("Opening Image", opening)

# Save output
cv2.imwrite("opening_output.jpg", opening)

cv2.waitKey(0)
cv2.destroyAllWindows()
```



Result

Thus, the **opening operation** was successfully performed using OpenCV.

Experiment 32 – Morphological Operations Using Closing Technique

Aim

To perform morphological **closing** on an image using OpenCV.

Procedure

1. Import OpenCV.
2. Read the input image.
3. Create a structuring element.
4. Apply closing using `cv2.morphologyEx()`.
5. Display the original and closing images.

6. Save the output image.

Code



Result

Thus, the **closing operation** was successfully performed using OpenCV.

Experiment 33 – Morphological Gradient Technique

Aim

To perform a **morphological gradient** on an image using OpenCV.

Procedure

1. Import OpenCV.

2. Read the input image.
3. Create a structuring element.
4. Apply morphological gradient.
5. Display the original and gradient images.
6. Save the output image.

Code

```
exp33.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/cv exp33.py/exp33.py (3.12.3)
File Edit Format Run Options Window Help
import cv2

# Read input image
image = cv2.imread("input.jpg")

if image is None:
    print("Error: Image not found.")
    exit()

# Create kernel
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))

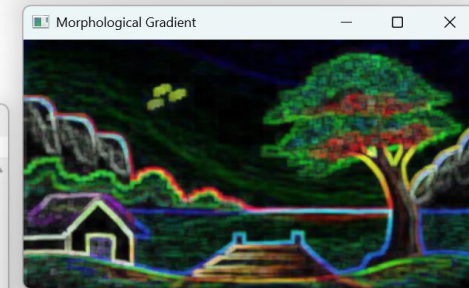
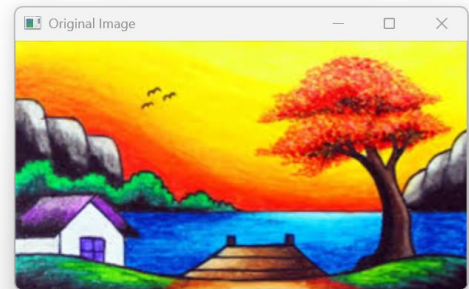
# Apply morphological gradient
gradient = cv2.morphologyEx(
    image,
    cv2.MORPH_GRADIENT,
    kernel
)

# Display images
cv2.imshow("Original Image", image)
cv2.imshow("Morphological Gradient", gradient)

# Save output
cv2.imwrite("gradient_output.jpg", gradient)

cv2.waitKey(0)
cv2.destroyAllWindows()

IDLE Shell 3.12.3
File Edit Shell Debug Options Window Help
Python 3.12.3 (tags/v3.12.3:f6650f9, Apr 9 2024, 14:05:25) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> = RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/cv exp33.py/exp33.py
```



Result

Thus, the **morphological gradient** was successfully performed using OpenCV.

Experiment 34 – Morphological Top Hat Technique

Aim

To perform **Top Hat morphological transformation** on an image using OpenCV.

Procedure

1. Import OpenCV.
2. Read the input image.
3. Create a structuring element.
4. Apply the Top Hat operation.
5. Display the original and Top Hat images.
6. Save the output image.

Code

```
exp34.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/cv exp34.py/exp34.py (3.12.3)
File Edit Format Run Options Window Help

import cv2

# Read input image
image = cv2.imread("input.jpg")

if image is None:
    print("Error: Image not found.")
    exit()

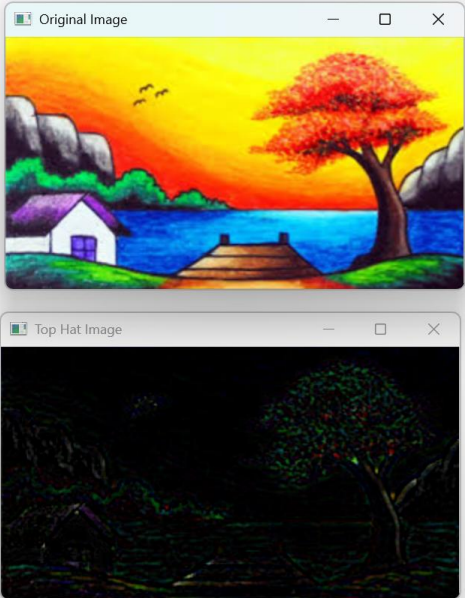
# Create kernel
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))

# Apply Top Hat transformation
top_hat = cv2.morphologyEx(
    image,
    cv2.MORPH_TOPHAT,
    kernel
)

# Display images
cv2.imshow("Original Image", image)
cv2.imshow("Top Hat Image", top_hat)

# Save output
cv2.imwrite("top_hat_output.jpg", top_hat)

cv2.waitKey(0)
cv2.destroyAllWindows()
```



The image shows two windows from a Python IDE. The 'Original Image' window displays a colorful landscape painting of a sunset over a body of water, with a small white house on the left and a large tree with red leaves on the right. The 'Top Hat Image' window displays the result of the Top Hat transformation, which is a dark image where the original image's features are highlighted in bright colors (red, green, blue) against a black background.

Result

Thus, the **Top Hat morphological transformation** was successfully performed using OpenCV.

Experiment 35 – Morphological Black Hat Technique

Aim

To perform **Black Hat morphological transformation** on an image using OpenCV.

Procedure

1. Import OpenCV.
2. Read the input image.
3. Create a structuring element.
4. Apply the Black Hat operation.
5. Display the original and Black Hat images.
6. Save the output image.

Code

```
exp35.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/cv exp35.py/exp35.py (3.12.3)
File Edit Format Run Options Window Help
import cv2

# Read input image
image = cv2.imread("input.jpg")

if image is None:
    print("Error: Image not found.")
    exit()

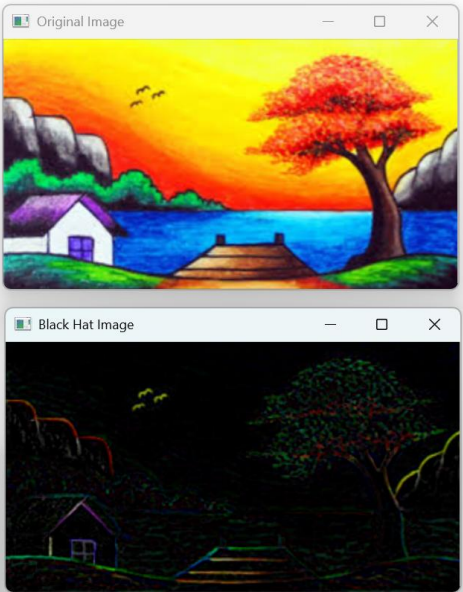
# Create kernel
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))

# Apply Black Hat transformation
black_hat = cv2.morphologyEx(
    image,
    cv2.MORPH_BLACKHAT,
    kernel
)

# Display images
cv2.imshow("Original Image", image)
cv2.imshow("Black Hat Image", black_hat)

# Save output
cv2.imwrite("black_hat_output.jpg", black_hat)

cv2.waitKey(0)
cv2.destroyAllWindows()
```



```
*IDLE Shell 3.12.3*
File Edit Shell Debug Options Window Help
Python 3.12.3 (tags/v3.12.3:f6650f9, Apr 9 2024, 14:05:25) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/cv exp35.py/exp35.py
```

Result

Thus, the **Black Hat morphological transformation** was successfully performed using OpenCV.

Experiment 36 – Recognize a Watch from an Image using Object Recognition

Aim

To recognize a **watch** from a given image using OpenCV-based object recognition.

Procedure

1. Import the required OpenCV libraries.
2. Load the pre-trained MobileNet-SSD object detection model.
3. Read the input image.
4. Detect objects present in the image.
5. Check whether the detected object is a watch.
6. Draw a bounding box around the detected object.
7. Display and save the result.

Code

```
exp36.py - C:\Users\Sravan Kumar.Y\AppData\Local\Programs\Python\Python312\cv exp36.py\exp36.py (3.12.3)
File Edit Format Run Options Window Help

import cv2
from ultralytics import YOLO

# Load pre-trained YOLO model
model = YOLO("yolo11n.pt")

# Read input image
image = cv2.imread("watch.jpg")

if image is None:
    print("Error: Image not found.")
    exit()

# Detect objects
results = model(image, verbose=False)

# Process detection results
for result in results:
    for box in result.boxes:
        confidence = float(box.conf[0])
        class_id = int(box.cls[0])
        label = model.names[class_id]
        if confidence > 0.5:
            x1, y1, x2, y2 = map(
                int,
                box.xyxy[0]
            )
            cv2.rectangle(
                image,
                (x1, y1),
                (x2, y2),
                (0, 255, 0),
                2
            )
            cv2.putText(
                image,
                f"{label} {confidence:.2f}",
                (x1, y1 - 10)
```

```
*IDLE Shell 3.12.3*
File Edit Shell Debug Options Window Help

Python 3.12.3 (tags/v3.12.3:f6650f9, Apr 9 2024, 14:05:25) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\Sravan Kumar.Y\AppData\Local\Programs\Python\Python312\cv exp36.py\exp36.py

Object Recognition

```

Ln: 5 Col: 0

Result

Thus, the given image was successfully processed using **YOLO-based object recognition with OpenCV**, and the detected objects were marked with bounding boxes.

Experiment 37 – Play Video in Reverse Mode using OpenCV

Aim

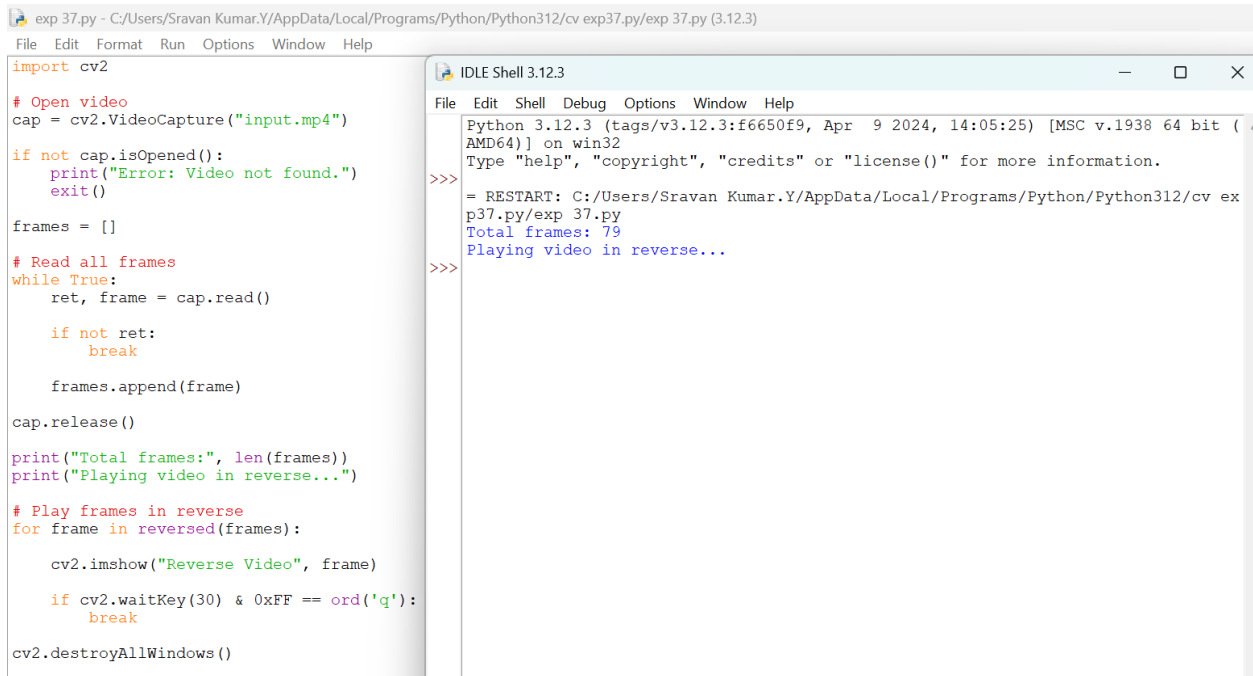
To read and play a video in **reverse mode** using OpenCV.

Procedure

1. Import OpenCV.
2. Open the input video.

3. Read all video frames.
4. Store the frames in a list.
5. Reverse the order of the frames.
6. Display the frames in reverse order.
7. Press q to stop the video.

Code



The screenshot shows a Python IDE with two windows. The main window displays a Python script for playing a video in reverse. The script opens a video file, reads all frames into a list, reverses the list, and then plays the frames in reverse order. It also includes a loop to wait for the 'q' key to be pressed before exiting. The second window, titled 'IDLE Shell 3.12.3', shows the output of the script: 'Total frames: 79' and 'Playing video in reverse...'.

```
import cv2

# Open video
cap = cv2.VideoCapture("input.mp4")

if not cap.isOpened():
    print("Error: Video not found.")
    exit()

frames = []

# Read all frames
while True:
    ret, frame = cap.read()

    if not ret:
        break

    frames.append(frame)

cap.release()

print("Total frames:", len(frames))
print("Playing video in reverse...")

# Play frames in reverse
for frame in reversed(frames):
    cv2.imshow("Reverse Video", frame)

    if cv2.waitKey(30) & 0xFF == ord('q'):
        break

cv2.destroyAllWindows()
```

```
Python 3.12.3 (tags/v3.12.3:f6650f9, Apr 9 2024, 14:05:25) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/cv exp37.py/exp 37.py
Total frames: 79
Playing video in reverse...
>>>
```

Result

Thus, the given video was successfully played in **reverse mode** using OpenCV.

Experiment 38 – Face Detection using OpenCV

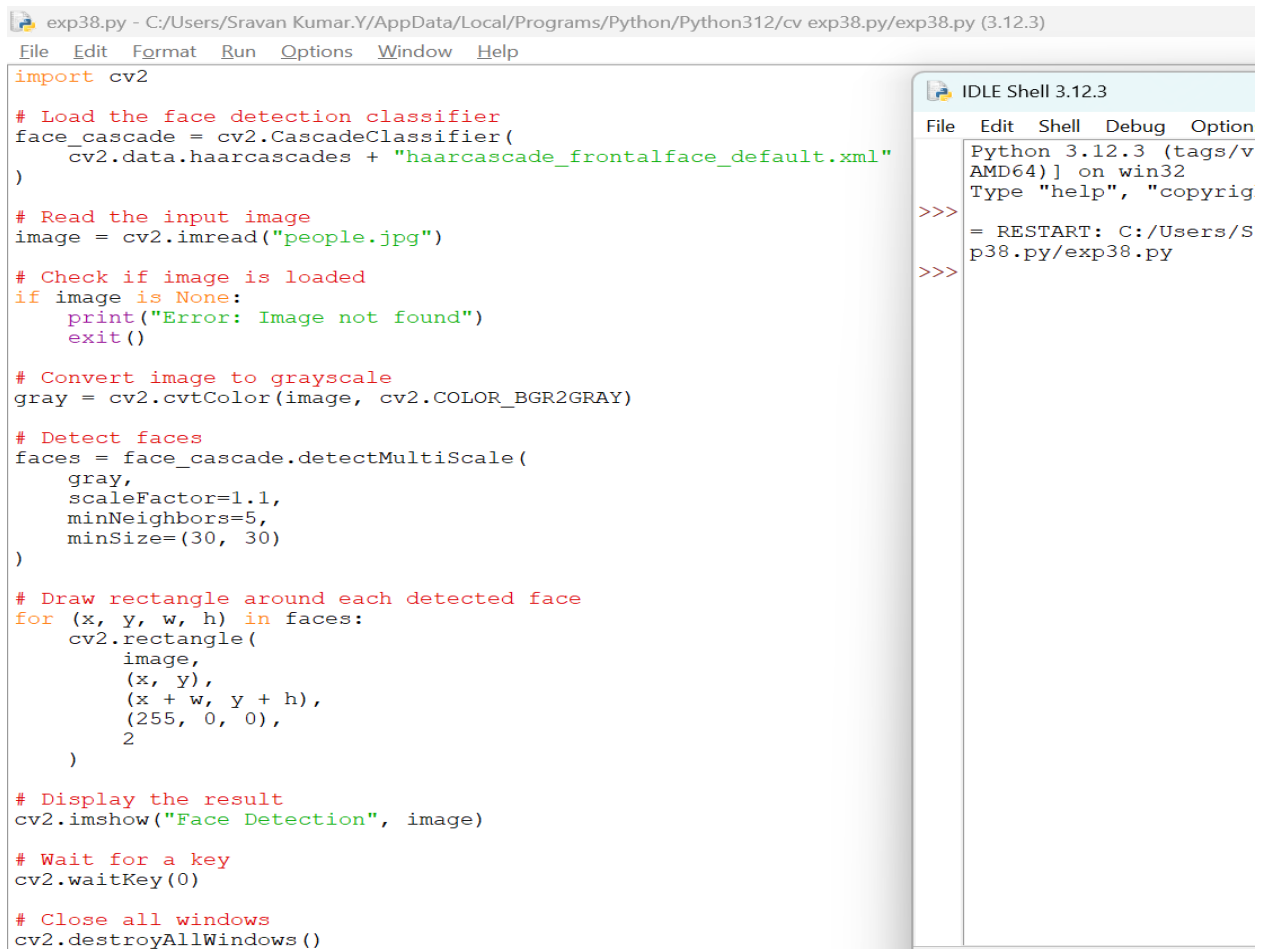
Aim

To detect human faces in an image using the **Haar Cascade classifier** in OpenCV.

Procedure

1. Import OpenCV.
2. Load the Haar Cascade face detector.
3. Read the input image.
4. Convert the image into grayscale.
5. Detect faces using detectMultiScale().
6. Draw rectangles around detected faces.
7. Display and save the result.

Code



```
exp38.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/cv exp38.py/exp38.py (3.12.3)
File Edit Format Run Options Window Help
import cv2

# Load the face detection classifier
face_cascade = cv2.CascadeClassifier(
    cv2.data.harcascades + "haarcascade_frontalface_default.xml"
)

# Read the input image
image = cv2.imread("people.jpg")

# Check if image is loaded
if image is None:
    print("Error: Image not found")
    exit()

# Convert image to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Detect faces
faces = face_cascade.detectMultiScale(
    gray,
    scaleFactor=1.1,
    minNeighbors=5,
    minSize=(30, 30)
)

# Draw rectangle around each detected face
for (x, y, w, h) in faces:
    cv2.rectangle(
        image,
        (x, y),
        (x + w, y + h),
        (255, 0, 0),
        2
    )

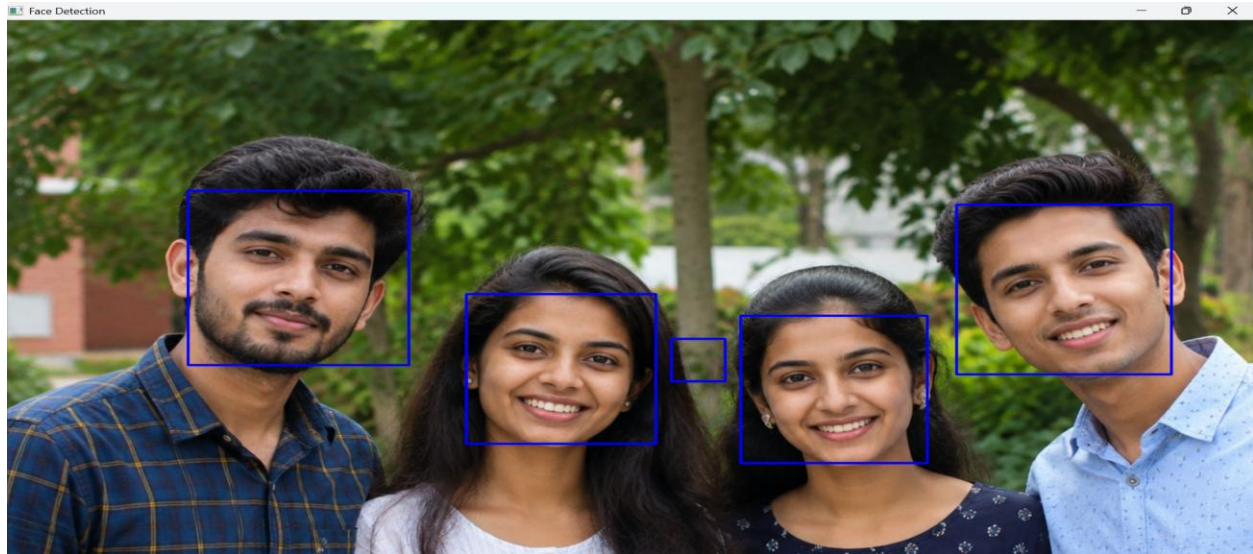
# Display the result
cv2.imshow("Face Detection", image)

# Wait for a key
cv2.waitKey(0)

# Close all windows
cv2.destroyAllWindows()
```

IDLE Shell 3.12.3

File	Edit	Shell	Debug	Option
Python 3.12.3 (tags/v				
AMD64)] on win32				
Type "help", "copyrig				
>>>				
				= RESTART: C:/Users/S
>>>				p38.py/exp38.py



Result

Thus, human faces were successfully detected using **OpenCV Haar Cascade**, and rectangular bounding boxes were drawn around the detected faces

Experiment 39: Vehicle Detection in a Video Frame Using OpenCV

Aim

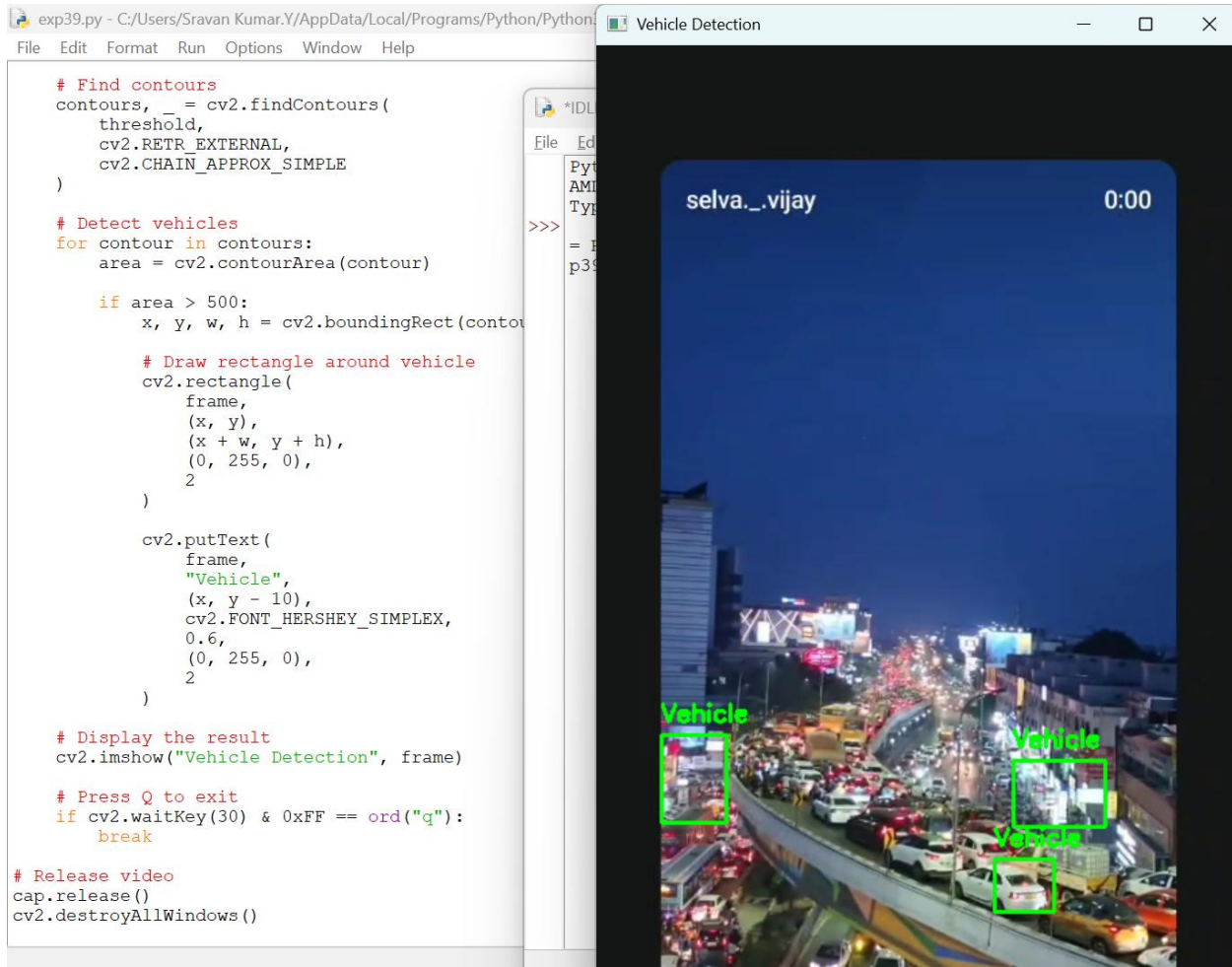
To detect vehicles in a video using OpenCV and display a rectangular bounding box around each detected vehicle.

Procedure

1. Import the OpenCV library.
2. Load the vehicle detection classifier.
3. Open the input video.

4. Read the video frame by frame.
5. Convert each frame to grayscale.
6. Detect vehicles in each frame.
7. Draw rectangles around detected vehicles.
8. Display the processed video.
9. Press **Q** to stop the program.
10. Release the video and close all windows.

Code



Result

Thus, vehicles were successfully detected from the video using OpenCV and rectangular bounding boxes were displayed around the detected vehicles.

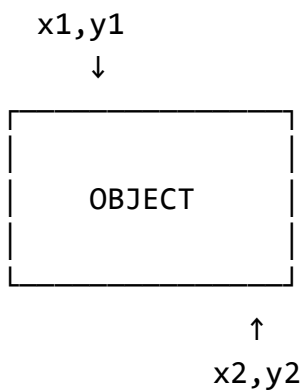
Experiment 40: Draw Rectangular Shape and Extract Objects

Aim

To draw a rectangular shape around an object and extract the selected object from an image using OpenCV.

Principle

A rectangular **Region of Interest (ROI)** is selected from the image using its coordinates.



The selected region is extracted using NumPy slicing.

Procedure

1. Import OpenCV.
2. Read the input image.
3. Define the coordinates of the object.
4. Draw a rectangle around the object.
5. Extract the rectangular ROI.
6. Display the original image with the rectangle.
7. Display the extracted object.
8. Save both outputs.

9. Close the windows.

Code

```
exp40.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/cv exp40.py/exp40.py (3.12.3)
File Edit Format Run Options Window Help

import cv2

# Read input image
image = cv2.imread("object.jpg")

if image is None:
    print("Error: Image not found.")
    exit()

# Rectangle coordinates
x1 = 100
y1 = 80
x2 = 300
y2 = 280

# Make a copy for marked image
marked_image = image.copy()

# Draw rectangle
cv2.rectangle(
    marked_image,
    (x1, y1),
    (x2, y2),
    (0, 255, 0),
    2
)

# Extract object / ROI
extracted_object = image[y1:y2, x1:x2]

# Display
cv2.imshow("Rectangle Around Object", marked_image)
cv2.imshow("Extracted Object", extracted_object)

# Save output
cv2.imwrite("rectangle_output.jpg", marked_image)
cv2.imwrite("extracted_object.jpg", extracted_object)

print("Object extracted successfully.")

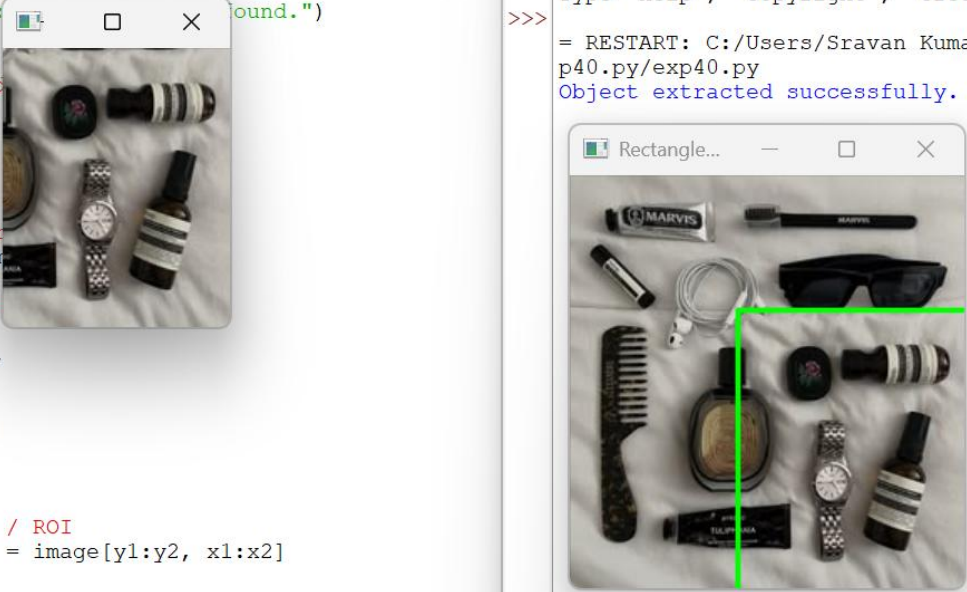
# Wait for key
cv2.waitKey(0)

# Close windows
cv2.destroyAllWindows()
```

```
*IDLE Shell 3.12.3*
File Edit Shell Debug Options Window Help

Python 3.12.3 (tags/v3.12.3:f6650f9, May 10 2024) on win32
Type "help", "copyright", "credits" or "quit()" for more.

>>>
= RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/Python312/python.exe C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/cv exp40.py/exp40.py
Object extracted successfully.
```



Result

Thus, a rectangular region was successfully drawn around the required object and the object was extracted from the image using OpenCV.

